

Cujae

Universidad Tecnológica de La Habana José Antonio Echeverría

Facultad de Ingeniería Informática

Informe de Prácticas Profesionales 2

Desarrollo de una nueva versión de TeamSoft+ incorporando factores demográficos religión y grupo de edades garantizando escalabilidad, usabilidad y eficiencia

Autores: Leyder Ignacio Herrera Bello

Dennis Serafín Rodríguez Armenteros

Tutores: Dr. C. Margarita André Ampuero

MCs. Tony García Álvarez

La Habana, Cuba

Octubre, 2025

Resumen

La formación de equipos es un proceso crítico para el éxito de las organizaciones, que requiere considerar múltiples factores individuales y colectivos. Teamsoft+ es una herramienta de apoyo a la toma de decisiones desarrollada en la Universidad Tecnológica de La Habana, que modela este proceso como un problema de optimización multiobjetivo. Si bien versiones anteriores de la herramienta han incorporado diversos factores, carecía de la capacidad de gestionar variables demográficas clave como la religión y la edad, elementos identificados en la literatura como cruciales para fomentar la armonía, el balance y la productividad en los equipos.

Este trabajo presenta el proceso de desarrollo y refactorización de Teamsoft+, el cual se ejecuta en dos etapas fundamentales. Primero, se amplían las capacidades del sistema mediante la incorporación de la gestión de los factores demográficos "religión" y "grupo de edades", permitiendo la formación de equipos con criterios de homogeneidad, heterogeneidad o balance respecto a estos. Esto implicó el rediseño de la base de datos, la implementación de nuevas funcionalidades en la interfaz de usuario y la generación de reportes avanzados. Segundo, para solucionar problemas de deuda técnica, usabilidad y escalabilidad, se ejecuta una migración completa de la herramienta hacia una arquitectura de Software como Servicio (SaaS). Este proceso incluyó la refactorización del código backend para eliminar duplicidades, la separación de la lógica de presentación y lógica de negocio mediante el desarrollo de una API RESTful, y la modernización del frontend para garantizar una experiencia de usuario intuitiva y accesible.

La solución fue validada mediante pruebas funcionales que confirmaron el correcto funcionamiento de las nuevas características. Como resultado, se obtuvo una versión de Teamsoft+ más robusta, abarcadora y escalable, que no solo enriquece el proceso de formación de equipos con nuevos criterios demográficos, sino que también se posiciona como una solución moderna y lista para su explotación comercial bajo un modelo SaaS.

Palabras claves: formación de equipos, Teamsoft+, factores demográficos, religión, grupo de edades, refactorización, Software como Servicio (SaaS), usabilidad, escalabilidad.

Abstract

Team building is a critical process for the success of organizations, which requires considering multiple individual and collective factors. Teamsoft+ is a decision-support tool developed at the Technological University of Havana, which models this process as a multi-objective optimization problem. While previous versions of the tool have incorporated a variety of factors, it lacked the ability to manage key demographic variables such as religion and age, elements identified in the literature as crucial to fostering harmony, balance, and productivity in teams.

This paper presents the Teamsoft+ development and refactoring process, which is executed in two fundamental stages. First, the system's capabilities are expanded by incorporating the management of the demographic factors "religion" and "age group", allowing the formation of teams with criteria of homogeneity, heterogeneity or balance with respect to these. This involved the redesign of the database, the implementation of new functionalities in the user interface and the generation of advanced reports. Second, to solve problems of technical debt, usability and scalability, a complete migration of the tool to a Software as a Service (SaaS) architecture is executed. This process included refactoring the backend code to eliminate duplications, separating the presentation and business logic by developing a RESTful API, and modernizing the frontend to ensure an intuitive and accessible user experience.

The solution was validated through functional tests that confirmed the correct functioning of the new features. As a result, a more robust, comprehensive and scalable version of Teamsoft+ was obtained, which not only enriches the team-building process with new demographic criteria, but also positions itself as a modern solution ready for commercial exploitation under a SaaS model.

Keywords: team building, Teamsoft+, demographics, religion, age group, refactoring, Software as a Service (SaaS), usability, scalability.

Índice

Introducción	1
Capítulo 1: Fundamentación teórica	8
1.1 Introducción	8
1.2 Problema de formación de equipo	8
1.2.1 Uso de las metaheurísticas para solucionar problemas de optimización... 10	
1.2.2 Uso de la biblioteca Biciam para dar solución al problema de formación de equipos..... 11	
1.3 Caracterización de la herramienta TeamSoft+. Potencialidades e insuficiencias. 12	
1.3.1 Deficiencias de la versión actual de Teamsoft+	14
1.3.2 La Refactorización como estrategia	16
1.4 Fundamentos del nuevo modelo de negocio	17
1.4.1 Modelos de distribución de software	18
1.4.2 Modelo de negocio de software híbrido	19
1.5 Fundamentación de las tecnologías a utilizar para el desarrollo de la aplicación 21	
1.5.1 Framework back-end..... 22	
1.5.2 Framework de front-end..... 23	
1.5.3 Gestor de base de datos	24
1.5.4 Herramientas de mapeo relacional	25
1.5.5 Spring Security..... 26	
1.5.6 SonarQube..... 26	
1.6 Conclusiones parciales..... 26	
Capítulo 2: Solución propuesta	27
2.1 Introducción	27
2.2 Incorporación de nuevas funciones objetivo	28
2.2.1 Funciones objetivo asociadas al factor religión y grupo de edad..... 28	
2.2.2 Modificaciones en la base de datos..... 29	
2.3 Rediseño de la arquitectura	31
2.4 Implementación de la APIREST	34

2.4.1 Diseño de los <i>endpoints</i> REST	34
2.4.2 Estructura modular	37
2.4.3 Patrones de diseño	38
2.5 Conclusiones parciales.....	41
Capítulo 3:	43
3.1 Introducción	43
3.2 Diseño y Ejecución de pruebas para validar el cálculo de los coeficientes de las funciones objetivo implementadas	43
3.2.1 Comprobar el funcionamiento de las funciones objetivo relacionadas con el factor Religión.....	44
3.2.2 Comprobar el funcionamiento de las funciones objetivo relacionadas con el factor Religión (Balance)	47
3.2.3 Comprobar el funcionamiento de las funciones objetivo relacionadas con el factor Grupo de Edades	51
3.2.3 Comprobar el funcionamiento de las funciones objetivo relacionadas con el factor Grupo de Edades (Balance)	54
3.3 Refactorización y mejoras en la Calidad estática del código	59
3.3.1 Análisis con SonarQube	59
3.3.2 Técnicas de refactorización ejecutadas	60
3.3.3 Resultados del análisis Post-Refactorización.....	61
3.4 Diseño y ejecución de pruebas para validar el correcto funcionamiento de los <i>endpoints</i> desarrollados	61
3.5 Pruebas de rendimiento	65
3.6 Conclusiones parciales.....	66
Conclusiones	67
Recomendaciones.....	69
Referencias Bibliográficas.....	70

Índice de Tablas

Tabla 1: Caso de prueba del Caso de Uso Gestionar Grupo de Edad (Crear).	62
Tabla 2: Caso de prueba del Caso de Uso Gestionar Grupo de Edad (Editar).	63
Tabla 3: Caso de prueba del Caso de Uso Gestionar Grupo de Edad (Eliminar).	64
Tabla 4: Resultados de pruebas de rendimiento realizadas a ambas versiones de la aplicación.....	65

Índice de Figuras

Figura 1: Representación de una solución en el problema de formación de múltiples equipos	12
Figura 2: Diagrama de clases de BICIAM extendido de la nueva versión de la herramienta TeamSoft+.	29
Figura 3: Modificaciones en la base de datos.	30
Figura 4: Representación del patrón N-Capas con un enfoque basado en la reutilización	34
Figura 5: Postman UI - Grupo de controladores para la gestión de Religiones	35
Figura 6: Ejemplo de petición exitosa al actualizar un nivel de competencia.	36
Figura 7: Ejemplo de petición exitosa al crear un nivel de competencia.....	36
Figura 8: Ejemplo de petición errónea al crear un nivel de competencia.....	36
Figura 9: Ejemplo de petición para un nivel de competencia que no se encuentra en la base de datos.....	37
Figura 10: Estructura modular de la nueva versión de TeamSoft+.	37
Figura 11: Representación del patrón Inyección de Dependencia en el proyecto.	39
Figura 12: Representación del patrón DTO en el proyecto (Diagrama UML).	40
Figura 13: Representación del patrón DTO en el proyecto (flujo de mapeo y persistencia). ...	40
Figura 14: Representación del patrón Singleton en el proyecto.	41
Figura 15: Propuesta de equipo heterogéneo (Factor Religión).	45
Figura 16: Propuesta de equipo homogéneo (Factor Religión).	46
Figura 17: Propuesta de equipos balanceados tomando en cuenta el factor Religión (Heterogéneo).	48
Figura 18: Propuesta de equipos balanceados tomando en cuenta el factor Religión (Homogéneo)	50
Figura 19: Propuesta de equipo Heterogéneo para el factor Grupo de Edades.....	52
Figura 20: Propuesta de equipo Homogéneo para el factor Grupo de Edad.	53
Figura 21: Propuestas de equipos balanceados tomando en cuenta el factor Grupo de Edad (Heterogéneo).	56
Figura 22: Propuesta de equipos balanceados tomando en cuenta el factor Grupo de Edad (Homogéneo).	58
Figura 23: Vista general de las métricas arrojadas por SonarQube de la versión anterior de TeamSoft+	60
Figura 24: Vista general de las métricas arrojadas por SonarQube en el código bak-end de la nueva versión de TeamSoft+.	61

Introducción

En la actualidad se reconoce el papel que juegan los equipos en el éxito de las organizaciones. Sin embargo, aún persiste un alto índice de fracasos en proyectos debido a factores humanos, como la inadecuada asignación de personal a los equipos de trabajo [1]. Este problema se observa en diversos contextos, donde la conformación de equipos suele realizarse de manera empírica y sin la incorporación de herramientas que faciliten una toma de decisiones fundamentada. La formación adecuada de equipos sigue siendo un desafío crítico que afecta la eficiencia y el éxito en una amplia gama de disciplinas. [2, 3]

El proceso de formación de equipos es complejo, ya que requiere tomar en cuenta varios factores [4, 5], algunos son de carácter individual como: las competencias necesarias para desempeñar determinado rol, la disponibilidad de las personas (relativa a su carga de trabajo) y sus preferencias por jugar determinado rol. Otros son factores colectivos como la presencia y el balance de las características psicológicas de las personas que forman el equipo y la incompatibilidad entre los miembros [6]. Esta gran cantidad y diversidad de factores hace que el proceso de formación sea muy difícil de realizar sin la ayuda de herramientas de apoyo a la toma de decisiones, acudiéndose en esos casos a la intuición o a la experiencia. La situación se vuelve más compleja en entornos donde la cantidad de personas a analizar y la cantidad de roles a cubrir es especialmente grande, dado que aumenta la cantidad de combinaciones a evaluar al ubicar a las personas disponibles en los roles del o de los equipos que se deben formar.

El problema de la formación de equipos ha sido abordado de diferentes maneras: Inicialmente, como un problema de asignación de personas a tareas o roles [1] teniendo en cuenta solo factores individuales y luego, como un problema de formación de equipos [2] considerando tanto factores individuales como colectivos. El concepto de equipo durante mucho tiempo se manejó como grupo de personas, sin tener en cuenta aspectos como la compatibilidad entre sus miembros, la capacidad para trabajar en conjunto y el equilibrio o presencia de diferentes perfiles psicológicos.

En el año 2009 en la Universidad Tecnológica de La Habana (CUJAE) se elaboró un modelo para la Asignación de Recursos Humanos a Equipos de Proyecto de Software [3, 7, 8] que

considera varios factores a la hora de formar equipos: las competencias necesarias para desempeñar determinado rol en el proyecto, la carga de trabajo de los miembros del equipo y las incompatibilidades entre los miembros. Se incluyó además el costo de desarrollo a distancia, para el caso de entidades que utilizan esta forma de trabajo. Como parte del trabajo se desarrolló la herramienta Teamsoft+ [9] para dar soporte al modelo.

A pesar de que el modelo citado fue definido inicialmente para formar un único equipo en el contexto del desarrollo de software, puede ser generalizable a múltiples contextos, siempre que se definan los roles y las competencias. Como aspecto innovador, incorpora elementos asociados a las características psicológicas en la formación del equipo como un todo, tema cuya importancia ha sido tratada en varios trabajos. [4, 6, 10, 11]

En aquel momento, el método empleado para la conformación de múltiples equipos consistía en formar los equipos de uno en uno (de forma secuencial). Sin embargo, este enfoque presentaba varias limitaciones. Los primeros equipos en formarse tendían a estar compuestos por las personas más competentes, mientras que los últimos equipos quedaban integrados por aquellos con menores habilidades, lo que generaba un desequilibrio notable. Además, existía la posibilidad de que algunos equipos no llegaran a formarse debido a la falta de personas con las competencias necesarias para cubrir los roles requeridos. Estas deficiencias subrayaban la necesidad de implementar estrategias más equitativas y eficaces que garantizaran una distribución de todos los equipos.

Ante esta situación se decide analizar los trabajos relacionados para identificar una propuesta de modelo y herramienta que permitiera formar uno o varios equipos considerando diversos factores y que fuera aplicable en diferentes contextos.

Durante la última década la formación de equipos se tornó un tema de investigación muy activo, incrementándose la cantidad de trabajos incluso en diferentes contextos como son: la formación de equipos multiagentes [5, 12-15], la formación de equipos de expertos en las redes sociales [16-23] y la formación de equipos de estudiantes [24-28]. Este auge provocó que se identificara la formación de equipos como un nuevo problema de optimización [2], ya que antes era visto como un problema de asignación.

A partir de investigaciones desarrolladas y el análisis de herramientas para la formación de equipos [29-33], se decide desarrollar una nueva versión de la herramienta TeamSoft+.

Durante los últimos cuatro años se han ido desarrollando varias versiones de la herramienta [34-37] lo cual ha permitido que la versión actual permita formar equipos considerando competencias tanto para un único rol como para desempeñar múltiples roles, interés por desempeñar un rol, los tipos de roles de Belbin en un equipo, la carga de trabajo, las incompatibilidades entre miembros, el costo de trabajar a distancia, la gestión de roles y usuarios, factores demográficos como nacionalidad y sexo y el balance de los equipos teniendo en cuenta estos factores.

El desarrollo de las diferentes versiones ha permitido que la herramienta Teamsoft+ pueda ser empleada por los decisores para formar equipos en diferentes contextos. Como parte de estos trabajos se evaluó el desempeño de la herramienta para formar equipos de proyectos de software, equipos de pelota, equipos de profesores para cubrir la carga docente y grupos de alumnos de 1er y 2do año de la Ing. Informática.

A pesar de que factores colectivos como la edad y la religión han sido identificados por diversos autores [38, 39] como elementos clave para fomentar la armonía en equipos de trabajo y garantizar un equilibrio que impulse la productividad en distintos entornos, estos aspectos no han sido considerados en versiones anteriores. La omisión de estos factores puede afectar la dinámica grupal, ya que la diversidad etaria contribuye a una combinación de experiencia e innovación, mientras que el respeto por las creencias religiosas fortalece la inclusión y el sentido de pertenencia. Incorporar estas variables en el análisis de equipos de trabajo permite una gestión más efectiva, promoviendo la colaboración y la cohesión entre sus miembros.

Adicionalmente, la herramienta carece de reportes que proporcionen una visión integral sobre todas las características específicas de las personas, ya que solo considera preferencias por roles, preferencias por equipos y competencias. Esto deja de lado aspectos clave como el tipo de rol según el modelo MBTI y las características del rol según la teoría de Belbin. Incluir estos elementos permitiría a los directivos tomar decisiones más informadas en la formación de equipos, optimizando la asignación de roles y mejorando el desempeño colectivo.

Otra limitación identificada en la herramienta es la falta de coherencia en su interfaz de usuario. Se observan discrepancias en el diseño de los botones, ausencia de información en secciones clave y variaciones en la estructura y presentación de las diferentes ventanas del programa. Estas inconsistencias afectan la experiencia del usuario, reduciendo la intuitividad y dificultando la navegación dentro del sistema.

Paralelamente, la arquitectura actual de Teamsoft+ presenta importantes limitaciones. Con el paso del tiempo y la participación de distintos desarrolladores, principalmente estudiantes, se ha acumulado una deuda técnica significativa, manifestada en la duplicación de código, una interfaz de usuario poco intuitiva y una estructura monolítica donde la interfaz de usuario y lógica del negocio se encuentran estrechamente acoplados. Esta situación dificulta enormemente el mantenimiento, la incorporación de nuevas funcionalidades y, sobre todo, la escalabilidad y el despliegue eficiente para múltiples clientes, imposibilitando un modelo de negocio de Software como Servicio (SaaS).

A partir la problemática descrita anteriormente se identifica como **problema a resolver** que la versión actual de TeamSoft+ no permite no permite formar equipos tomando en cuenta factores demográficos religión y grupo de edades, presenta problemas de usabilidad y calidad estática del código y su arquitectura dificulta su transición hacia un modelo SaaS que facilite su escalabilidad y despliegue eficiente para múltiples clientes.

Para dar solución al problema se define como **objeto de estudio** la creación de aplicaciones SaaS y como **campo de acción**, creación de TeamSoft+ como aplicación SaaS.

Como **objetivo general** se ha decidido desarrollar una nueva versión de TeamSoft+ convirtiéndola en una plataforma SaaS que incorpore los factores demográficos de religión y grupo de edades para la formación de equipos.

Para dar cumplimiento al objetivo general se plantean los siguientes **objetivos específicos y tareas**:

- 1) Caracterizar la versión actual de la herramienta Teamsoft+, precisando sus insuficiencias.
 1. Estudiar el problema de formación de equipo y la versión actual de TeamSoft+ (Leyder y Dennis).

2. Identificar las insuficiencias en la herramienta actual respecto a la falta de factores demográficos (Leyder y Dennis).
 3. Analizar problemas de usabilidad en la interfaz gráfica existente (Leyder).
 4. Identificar limitaciones técnicas que dificultan el mantenimiento y escalabilidad (Leyder y Dennis).
 5. Identificar limitaciones técnicas que dificultan el mantenimiento y escalabilidad (Leyder y Dennis).
- 2) Fundamentar la selección de las tecnologías a utilizar en el desarrollo de la nueva versión de la herramienta.
1. Investigar y comparar *frameworks* de *front-end* considerando rendimiento, curva de aprendizaje y ecosistema de desarrollo (Leyder y Dennis).
 2. Analizar *frameworks back-end* para aplicaciones Java evaluando configuración, productividad (Leyder y Dennis).
 3. Evaluar sistemas de gestión de bases de datos relacionales (Leyder y Dennis).
 4. Analizar metodologías de desarrollo y arquitecturas utilizadas para el desarrollo de servicios en la nube (Leyder y Serafín).
- 3) Dotar a la herramienta de la posibilidad de formar equipos considerando factores demográficos como religión y grupo de edades.
1. Diseñar funciones objetivo para formación de equipos homogéneos/heterogéneos en religión (Dennis).
 2. Diseñar funciones objetivo para formación de equipos homogéneos/heterogéneos en grupo de edades (Leyder).
 3. Diseñar las funciones objetivo para el balance de religión y grupo de edades (Leyder y Dennis).
 4. Crear tablas para los nuevos nomencladores (religión y grupo de edades) (Dennis).
 5. Modificar la interfaz para permitir seleccionar criterios de religión y edad en formación de equipos (Leyder).
- 4) Rediseñar la arquitectura para la nueva versión de TeamSoft+.
1. Realizar ajustes al diseño físico de la base de datos para soportar el nuevo modelo (Leyder y Dennis).

2. Documentar las tablas y campos modificados en la base de datos (Leyder y Dennis).
 3. Realizar la migración de los datos existentes hacia la nueva base de datos (Dennis).
- 5) Desarrollar la API para lograr una conexión satisfactoria entre el cliente y las funcionalidades.
1. Implementar la gestión de nomencladores (religión, grupo de edades, competencias, roles, en total 17 nomencladores) (Leyder y Dennis).
 2. Implementar gestión de personas (Dennis).
 3. Implementar gestión de equipos (Dennis).
 4. Implementar lógica para el cálculo de funciones objetivo con los nuevos factores (Leyder y Dennis).
- 6) Validar la solución propuesta.
1. Diseñar pruebas para validar el cálculo correcto de coeficientes de las funciones objetivo de religión (Dennis).
 2. Diseñar pruebas para validar el cálculo correcto de coeficientes de las funciones objetivo de grupo de edad (Leyder).
 3. Ejecutar pruebas y documentar resultados del rendimiento de la API desarrollada (Leyder y Dennis).
 4. Diseñar casos de prueba para las funcionalidades implementadas (Operaciones de Insertar, Crear, Modificar, Eliminar nomencladores) (Leyder y Dennis).
 5. Ejecutar casos de prueba para las funcionalidades implementadas (Leyder y Dennis).
 6. Validar usabilidad mediante pruebas de experiencia de usuario (Leyder y Dennis).

El presente documento se estructura en tres capítulos. El Capítulo 1 establece el marco teórico, analizando la herramienta TeamSoft+, identificando sus insuficiencias, se detallan los fundamentos de la formación de equipos, se ofrecen definiciones del modelo de Software como Servicio y la arquitectura *single-tenant/multi-tenant*, así como sus características, ventajas y desventajas. Por último, se fundamentan las tecnologías a utilizar para el desarrollo de la solución.

El Capítulo 2 detalla la solución propuesta, describiendo el diseño de la nueva arquitectura, las modificaciones en la base de datos, la implementación de los nuevos factores demográficos y el proceso de refactorización , la estructura del proyecto de *front-end* y *back-end*, patrones de diseño y arquitectónicos usados, las vistas y componentes implementados con respecto a la creación de los equipos, y se explican las mejoras de usabilidad en la interfaz de la herramienta con respecto a la anterior interfaz..

El Capítulo 3 presenta la validación de la solución a través de pruebas funcionales, de usabilidad y de rendimiento, demostrando la efectividad de la nueva versión de Teamsoft+.

Capítulo 1: Fundamentación teórica

1.1 Introducción

La gestión efectiva de equipos de trabajo representa uno de los desafíos más significativos en el cualquier tipo de institución contemporánea. TeamSoft+, como herramienta especializada en la formación de equipos, enfrenta limitaciones conceptuales y técnicas que restringen su capacidad para optimizar la composición de equipos en entornos diversos y multiculturales. Este capítulo establece los fundamentos teóricos que sustentan la necesidad de evolucionar hacia un modelo de Software como servicio (SaaS, por sus siglas) que incorpore factores de diversidad críticos como la religión y los grupos de edades, aspectos que la literatura reciente identifica como determinantes en la configuración de las "líneas de falla" que afectan la cohesión y efectividad de los equipos.

Este capítulo analiza sistemáticamente los componentes teóricos esenciales para esta transformación, estableciendo las bases para el desarrollo de una solución robusta y contextualmente sensible.

1.2 Problema de formación de equipo

El problema de formación de equipos consiste en asignar personas a roles en un equipo teniendo en cuenta diversos factores, tales como las competencias, la carga de trabajo, las relaciones entre los miembros, así como factores demográficos como religión, edad, sexo, nacionalidad, entre otros. Algunos de estos factores pueden generar líneas de fallas (en inglés, *faultlines*). En el contexto organizacional, el concepto de "línea de falla" se refiere a una fractura estructural interna de naturaleza psicosocial. La Teoría de las Líneas de Falla, introducida por Lau y Murnighan (1998) [40], conceptualiza la diversidad no como una simple mezcla de atributos (edad, género, experiencia), sino como la alineación hipotética de múltiples atributos que tienen el potencial de dividir a un equipo en subgrupos o coaliciones cohesivas [40-42]. Es esta alineación, más que la diversidad absoluta, la que predice el conflicto interno, la polarización y, en última instancia, el bajo rendimiento del equipo.

Los factores a considerar varían en dependencia del contexto donde se desee formar el o los equipos [42]. La gran cantidad y diversidad de factores hace que el proceso de formación sea difícil de realizar sin la ayuda de herramientas de apoyo a la toma de decisiones, acudiéndose en esos casos a la intuición o a la experiencia. La situación se vuelve más compleja en organizaciones grandes, con mayor cantidad de personas disponibles dado que aumenta la cantidad de combinaciones a evaluar al ubicar a las personas disponibles en los roles del equipo que se debe formar [37].

En el año 2009 en la Universidad Tecnológica de La Habana se elaboró un modelo [3] para asignar recursos humanos a equipos de proyecto de software y posteriormente, se desarrolló una herramienta llamada Teamsoft+ [19] que permitió darle soporte. La herramienta permite configurar los parámetros usados en funciones objetivo y restricciones del modelo, y definir las competencias y roles en función de la organización donde se desee aplicar. Estas facilidades de configuración ayudan a que la herramienta pudiera ser aplicada en múltiples contextos, a pesar de desarrollarse para formar equipos de proyectos de software [37].

Sin embargo, tanto el modelo como la herramienta se enfocaban en la formación de un solo equipo. Esto limitaba su uso cuando se deseaban formar múltiples equipos como, por ejemplo: equipos de estudiantes para desarrollar proyectos de curso, equipos quirúrgicos y equipos de linieros ante una contingencia. Utilizar la herramienta desarrollada para dar solución a las situaciones anteriores, implicaría formar los equipos de uno en uno (de forma secuencial). Como resultado los primeros equipos en formarse serían muy buenos y los últimos, podrían resultar malos. Dado que en cada ejecución de la herramienta se seleccionan los mejores candidatos disponibles, los equipos formados podrían quedar muy 10 desbalanceados, incluso podría ocurrir que no quedaran personas disponibles para cubrir los roles de los últimos equipos [37].

Ante esta situación se decide analizar los trabajos relacionados para identificar una propuesta de modelo y herramienta que permitiera formar uno o varios equipos considerando diversos factores y que fuera aplicable en diferentes contextos. Como parte del estudio se identificó que durante la última década el área de investigación relacionada con la formación de equipos se tornó más activa, en tanto se incrementaron las investigaciones asociadas a estas temáticas, en especial a partir de que fue identificada la formación de equipos como un nuevo problema de optimización [2, 37, 43]. En [40, 41, 44, 45] se identifican factores tanto individuales como

colectivos considerados en la formación de uno o varios equipos a partir de analizar un conjunto amplio de trabajos relacionados. Estos factores se listan a continuación:

Los factores individuales son:

- Competencias de las personas para desempeñar determinado rol o tarea.
- Experiencia en el desempeño de un rol o una tarea.
- Formación académica de la persona.
- Disponibilidad de la persona teniendo en cuenta la carga de trabajo.
- Costo de desarrollar el rol o la tarea a distancia.
- Características psicológicas.
- Interés de la persona por participar en un equipo.

Los factores colectivos son:

- Experiencia colaborando juntos.
- Diversidad demográfica en el equipo (en cuanto a sexo, nacionalidad, entre otros).
- Conflicto entre miembros del equipo.
- Balance entre las características psicológicas.
- Heterogeneidad en los tipos MBTI.

Sin embargo, aunque aumentó el número de trabajos y se identificaron mayor cantidad de factores a considerar en el proceso de formación de equipo, los modelos propuestos no eran abarcadores, se centraban cada uno en pocos factores, restringidos al contexto.

1.2.1 Uso de las metaheurísticas para solucionar problemas de optimización

El problema de formación de múltiples equipos dentro de una organización puede ser abordado desde una perspectiva de optimización combinatoria, dado que implica la evaluación de un amplio conjunto de combinaciones posibles de individuos para cubrir los distintos roles definidos en cada equipo. En este contexto, el proceso de toma de decisiones involucra múltiples factores o funciones objetivo, lo que convierte el problema en una instancia de optimización multiobjetivo.

Para abordar esta problemática, es necesario emplear técnicas que permitan obtener soluciones factibles y de alta calidad en tiempos razonables. En este sentido, los algoritmos metaheurísticos han demostrado ser una alternativa eficiente, ya que exploran de manera inteligente una fracción del espacio de búsqueda, sin necesidad de evaluar todas las combinaciones posibles.

En el proceso de conformación de equipos dentro de una entidad, se requiere seleccionar una metodología de optimización que permita evaluar múltiples criterios simultáneamente. Para ello, se pueden emplear dos enfoques principales: el método de factores ponderados y el multiobjetivo puro [46].

El método de factores ponderados convierte un problema multiobjetivo en un problema monobjetivo mediante la asignación de pesos a cada función objetivo. Posteriormente, estos valores se combinan en una única expresión matemática que permite calcular la solución final. Este enfoque resulta útil cuando se dispone de un criterio bien definido para establecer la prioridad de cada objetivo [47].

Por otro lado, el método multiobjetivo puro considera todas las funciones objetivo de manera simultánea, evaluando cada solución en función de su desempeño en cada criterio. Las soluciones que no superan a otras en al menos una función objetivo son descartadas y denominadas soluciones dominadas. En contraste, aquellas soluciones que no son superadas por ninguna otra en todos los criterios se agrupan como soluciones no dominadas y constituyen el conjunto de soluciones finales [47].

La selección entre estos métodos depende del contexto de aplicación y de las prioridades del decisor, quien deberá determinar la estrategia más adecuada para la formación de equipos con base en los requerimientos específicos de la entidad.

1.2.2 Uso de la biblioteca Biciam para dar solución al problema de formación de equipos

Existe una tendencia de usar bibliotecas de clases de algoritmos metaheurísticos en la solución de problemas de optimización. En el desarrollo de la herramienta Teamsoft+ se decidió emplear la biblioteca Biciam [46-50] ya que provee algoritmos para solucionar tanto problemas monobjetivo como multiobjetivo, además de métodos de solución de problemas multiobjetivo

como: factores ponderados, ordenamiento lexicográfico y multiobjetivo puro. Utilizar Biciam permite aprovechar la diversidad de algoritmos que ofrece la biblioteca para experimentar cuáles tienen mejor desempeño en la solución del problema en cuestión (en este caso, la formación de equipos) [46-48, 51-53].

Para implementar la solución en Teamsoft+ [54] se define una lista de elementos (equipos-roles), donde cada elemento está compuesto por una lista de roles-personas y cada lista de roles-personas contiene el rol y la lista de personas que juegan ese rol. La Figura 1 muestra una representación gráfica de una solución [37].

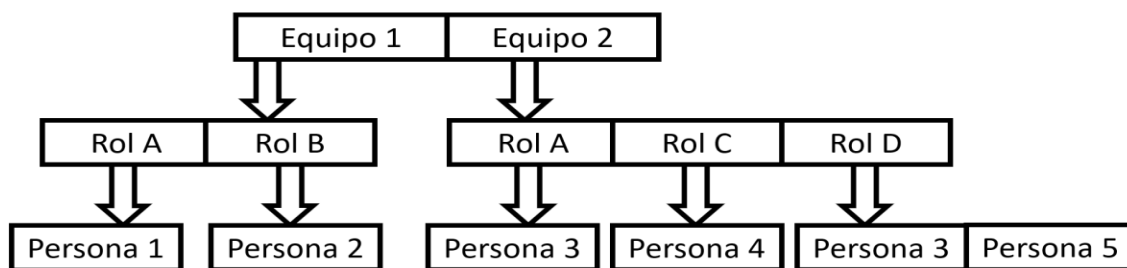


Figura 1: Representación de una solución en el problema de formación de múltiples equipos

1.3 Caracterización de la herramienta TeamSoft+. Potencialidades e insuficiencias.

En el año 2009 en la Universidad Tecnológica de La Habana se elaboró un modelo [3] para asignar recursos humanos a equipos de proyecto de software y posteriormente, se desarrolló una herramienta llamada Teamsoft+ [19] que permitió darle soporte. La herramienta permite configurar los parámetros usados en funciones objetivo y restricciones del modelo, y definir las competencias y roles en función de la organización donde se desee aplicar. Estas facilidades de configuración ayudan a que la herramienta pudiera ser aplicada en múltiples contextos, a pesar de desarrollarse para formar equipos de proyectos de software [37].

Sin embargo, tanto el modelo como la herramienta se enfocaban en la formación de un solo equipo. Esto limitaba su uso cuando se deseaban formar múltiples equipos como, por ejemplo: equipos de estudiantes para desarrollar proyectos de curso, equipos quirúrgicos y equipos de

linieros ante una contingencia. Utilizar la herramienta desarrollada para dar solución a las situaciones anteriores, implicaría formar los equipos de uno en uno (de forma secuencial). Como resultado los primeros equipos en formarse serían muy buenos y los últimos, podrían resultar malos. Dado que en cada ejecución de la herramienta se seleccionan los mejores candidatos disponibles, los equipos formados podrían quedar muy 10 desbalanceados, incluso podría ocurrir que no quedaran personas disponibles para cubrir los roles de los últimos equipos [37].

Ante esta situación se decide analizar los trabajos relacionados para identificar una propuesta de modelo y herramienta que permitiera formar uno o varios equipos considerando diversos factores y que fuera aplicable en diferentes contextos. Aunque aumentó el número de trabajos y se identificaron mayor cantidad de factores a considerar en el proceso de formación de equipo, los modelos propuestos no eran abarcadores, se centraban cada uno en pocos factores, restringidos al contexto.

Antes de desarrollar una nueva versión de la herramienta Teamsoft, se analizaron un grupo de herramientas empleadas en la formación de equipos buscando identificar si eran capaces de dar soporte al nuevo modelo definido. Entre ellas se encuentran: TeamFormationTool [14], Herramienta de soporte para la formación de grupos de trabajo en entornos de aprendizaje colaborativo [15]. Herramienta web para apoyar la formación de equipos en cursos de diseño Capstone [16], GoSim [17], DisTeam [18], TeamGen [19], Team Machine [55] y TeamBuilder [56]. El estudio permitió concluir que la mayoría de las herramientas se circunscribían al contexto académico, lo cual dificultaba su aplicación en otros ámbitos. Eran pocas las que consideraban los factores de equipo mencionados anteriormente, por lo que en esencia se formaban grupos y no equipos.

A partir de este análisis, se decide desarrollar una nueva versión de la herramienta Teamsoft+ [57, 58], que brinda soporte al nuevo modelo. En esta versión se incluyeron las funciones objetivo y restricciones relacionadas con las competencias, el interés por desempeñar los roles, la diversidad de roles de Belbin en el equipo, la carga de trabajo, las incompatibilidades entre los miembros, el costo de trabajar a distancia y el balance de los equipos teniendo en cuenta estos factores.

En el año 2021, se desarrolló una nueva versión de la herramienta [34, 35, 59], en la cual se incluyeron funciones objetivo y restricciones relacionadas con el interés por trabajar en el equipo

y la presencia de diversidad de tipos psicológicos basado en el test de Myers-Briggs. Además, se adicionaron funcionalidades que permiten configurar los parámetros de los algoritmos a utilizar y almacenar los resultados de la ejecución de los algoritmos, lo que facilita el desarrollo de experimentos.

En el año 2022 se actualizó la versión anterior, incorporando funciones objetivo para maximizar en el equipo las personas con competencias para jugar más de un rol y para balancear los equipos en función de este aspecto. Se adiciona la gestión de roles y usuarios, y la posibilidad de fijar personas en roles durante la formación de los equipos, lo que facilita el trabajo de los decisores [36].

En el año 2023 se amplía la herramienta incorporándole nuevas funciones objetivo para incluir en los equipos la posibilidad de tomar en cuenta factores demográficos como la nacionalidad y el sexo de las personas. Estos factores fueron utilizados para asegurar diversidad o alcanzar homogeneidad en los equipos que se formaban [37].

1.3.1 Deficiencias de la versión actual de Teamsoft+

El desarrollo de las diferentes versiones ha permitido que la herramienta Teamsoft+ pueda ser empleada por los decisores para formar equipos en diferentes contextos, ya que se pueden tomar en cuenta gran cantidad de factores a partir de las funciones objetivo implementadas. Sin embargo, factores demográficos como la edad y la religión, identificados en la bibliografía como elementos clave para fomentar la armonía y el equilibrio en los equipos, no están incorporados en la herramienta. Según la Teoría de las Líneas de Falla [38, 60], estos factores demográficos se reconocen como algunos de los que más afectan la subdivisión de equipos en subgrupos más homogéneos, donde los miembros comparten criterios afines. Cuando múltiples atributos como la edad y la religión se alinean, pueden crearse divisiones latentes dentro del equipo que debilitan la cohesión grupal y afectan negativamente la productividad. Por lo tanto, la incorporación de estos factores en Teamsoft+ resulta crucial para permitir a los decisores gestionar proactivamente estas potenciales líneas de fractura y formar equipos más integrados y efectivos en distintos entornos organizacionales. [40, 41, 45].

A pesar de todas las fortalezas y funcionalidades implementadas, la versión actual de Teamsoft+ aún presenta varias insuficiencias que limitan la calidad del proceso de formación de equipos, la usabilidad del sistema, su integridad y solidez. A continuación, se listan estas insuficiencias:

- Aunque la aplicación está implementada para tener en cuenta la mayoría de los factores colectivos, no tiene en cuenta la edad y la religión de las personas que puedan formar parte de los equipos.
- La interfaz actual no posibilita una navegación intuitiva y organizada.
- El código fuente de la aplicación presenta errores de duplicidad del código, haciendo más engorrosa su lectura y dificultando su escalabilidad.
- La herramienta carece de reportes que brinden la mayor información posible sobre las personas y permita su filtrado por cada una de ellas.
- La lógica de presentación se encuentra estrechamente relacionada con la lógica de negocio, lo que dificulta la capacidad de escalar fácilmente el software.

La herramienta presenta problemas con la calidad estática del código. Martin Fowler en [61] lo denomina como “olores de código”, haciendo referencia a “una indicación superficial que suele corresponder a un problema más profundo en el sistema”. Añade que los olores de código son “rápidos de detectar o rastrear” [61].

Dentro del catálogo de “malos olores” se identifican un conjunto de problemas que son indicadores que es necesario refactorizar [61]:

- Código Duplicado: El mismo código aparece en múltiples lugares.
- Función Larga: Funciones o métodos que exceden lo razonable en longitud. Múltiples niveles de anidación, por lo que se necesitan comentarios para explicar las secciones.
- Clase Grande: Clases que intentan hacer demasiado con excesiva cantidad de métodos. Que implica en general, una agrupación artificial de funcionalidades no relacionadas.
- Lista Larga de Parámetros: Funciones con excesivos parámetros, más de tres o cuatro parámetros por función. Dificultad para recordar el orden y propósito.

En Teamsoft+ estos “malos olores” se aplican a través de las diferentes etapas de desarrollo por las que ha atravesado el software, en la que cada etapa la integraban programadores diferentes. Esto trajo consigo problemas en la calidad estática del código, principalmente en la

cantidad de código duplicado, clases extremadamente grandes en la que varios desarrolladores participaron en su programación, clases y métodos duplicados e inutilizados en la aplicación.

1.3.2 La Refactorización como estrategia

La refactorización de código, definida como el proceso de mejora de las estructuras del código para mejorar la legibilidad y comprensión sin cambiar su funcionalidad. Martin Fowler popularizó esta práctica como mecanismo para eliminar "olores de código" que indican problemas subyacentes en el sistema [61].

La refactorización de código puede requerir varias técnicas de refactorización, según su objetivo. A continuación, se enumeran algunas de las técnicas más comunes de refactorización [61-63]:

- **Simplificación:** La simplificación implica refactorizar un fragmento de código en funciones más simples con un mínimo de código siempre que sea posible y eliminar las rutas no utilizadas o en desuso. La simplificación hace que el código sea más fácil de entender y más rápido de compilar y ejecutar.
- **Composición:** La composición divide un bloque de código con múltiples funciones en bloques discretos de una sola función. Estos bloques discretos se llaman en orden desde una parte separada del código. La mejora de la composición reduce los errores y mejora la reutilización del código.
- **Desglosamiento del código monolito:** Muchos bases de código complejas comienzan la vida como una sola aplicación monolítica, un solo proyecto de código o un conjunto estrechamente acoplado de módulos, todos mantenidos y lanzados juntos.

Con el tiempo, los monolitos pueden volverse difíciles de manejar, las construcciones tardan una eternidad, un cambio en un área puede afectar involuntariamente a otra, y los equipos pueden ser complejos a escala porque todos pisan los dedos de los pies en el mismo código. Un desafío común de refactorización para los ingenieros senior es modularizar o dividir un monolito en piezas más manejables

¿Cuáles son los beneficios de la refactorización de código?

El objetivo principal de la refactorización es hacer que el código sea más fácil de mantener en el futuro y luchar contra la deuda técnica. Refactorizamos porque entendemos que hacer el

diseño correcto a la primera es difícil y también obtiene los siguientes beneficios de la refactorización [61, 63, 64]:

- El tamaño del código a menudo se reduce.
- El código confuso se reestructura en un código más simple.
- Mejora el rendimiento del programa tanto en tiempo de ejecución como de compilación.
- Actualización a estándares modernos.

Para TeamSoft+, la refactorización permitirá:

1. Reducción de deuda técnica: Eliminación de duplicaciones, métodos extensos y código obsoleto que dificultan el mantenimiento y evolución.
2. Organización: Reorganización de la estructura de carpetas monolítica actual transformándola en una estructura modular en capas, ganando en organización y legibilidad.
3. Mejora de la mantenibilidad: Implementación de estándares de codificación y patrones de diseño que faciliten la incorporación de nuevos desarrolladores.

1.4 Fundamentos del nuevo modelo de negocio

Un modelo de negocio describe el plan de una empresa para generar ingresos, obtener beneficios y ofrecer valor a sus clientes. El modelo de negocio incluye todos los procedimientos y normas desarrollados por la dirección de la empresa para gestionar el negocio.

Existen varios tipos principales de modelos de negocio de software, cada uno con sus propias características únicas como se enumeran en [65]:

- Modelos de negocio de distribución: Son los más importantes en el sector del software, incluyen tres tipos: local, en la nube e híbrido. Estos modelos dependen de cómo una empresa de software desee distribuir sus productos y servicios a sus clientes.
- Modelos de negocio de licencias de software: Implican la concesión de licencias del código fuente que crea una empresa. Los dos tipos principales de modelos de licencia de código fuente son el propietario y el código abierto.

- Modelos de negocio de audiencia objetivo: Dependen de si una empresa de software quiere dirigirse a otras empresas (B2B) o a consumidores (B2C) al vender sus productos y servicios.
- Modelos de negocio de ingresos de software: Abarcan una amplia variedad, dependiendo de las fuentes de ingresos habituales con los suscriptores y las compras de aplicaciones.

1.4.1 Modelos de distribución de software

La elección de un modelo de distribución es una decisión estratégica fundamental, ya que define la arquitectura del sistema, la experiencia del usuario, el flujo de ingresos y la relación a largo plazo con el cliente. Para una herramienta como TeamSoft+, que transita de un entorno académico a un entorno comercial, comprender estos modelos es el primer paso para diseñar una solución viable y sostenible.

Tradicionalmente, el software se concebía como un producto tangible que se adquiría de forma perpetua. Sin embargo, el advenimiento de la nube y la conectividad omnipresente ha impulsado paradigmas donde el software se consume principalmente como un servicio. En este espectro que va desde el producto físico hasta el servicio puro en la nube, existen varios modelos, cada uno con sus propias características, ventajas e inconvenientes. A continuación, se analizan los modelos más relevantes para el contexto de este proyecto: el software on-premise tradicional, el Software como Servicio (SaaS) y el Software como Producto (SaaP), con el fin de sentar las bases que justificarán el modelo híbrido propuesto para TeamSoft+.

Modelo de negocio de software local (On-premise): El modelo de negocio de distribución de software local presupone que el software se instala en el hardware del cliente. Esto significa que usted compra un producto de software y recibe un kit de distribución para instalarlo en su servidor propio, donde se almacenarán todos los datos [65].

Modelo de negocio de software como servicio (SaaS): El SaaS es un modelo de negocio en el que un producto de software se ofrece como un servicio web mediante suscripción. Los clientes pueden usar el producto de inmediato, sin perder tiempo en la instalación. Los programas ya están configurados para diversas tareas empresariales [65, 66].

Modelo de negocio de software como producto (PaaS): Es un modelo de negocio que cambia la venta de productos físicos o digitales a un modelo de suscripción, donde el cliente paga por el uso del producto en lugar de poseerlo. El proveedor del producto es el encargado de darle soporte y mantenimiento, en cambio el cliente sólo se centra en utilizarlo [67].

Para desarrollar la nueva versión de Teamsoft+ será necesario llevar a cabo un nuevo modelo de negocio, convirtiéndolo de una herramienta convencional y un modelo de negocio de distribución híbrida ya que contará con el software como producto, pero a la vez con una gestión de servicio.

1.4.2 Modelo de negocio de software híbrido

El modelo de software híbrido, como su nombre indica, es un enfoque de distribución que combina aplicaciones de software locales con el uso de soluciones en la nube [68].

La ventaja del modelo de distribución de software híbrido radica en la mayor flexibilidad que ofrece a los clientes. Estos pueden trasladar información y datos entre sus centros locales y los servicios en la nube, sin depender exclusivamente de su propia infraestructura. Asimismo, la información confidencial, puede permanecer en las instalaciones del cliente, mientras que los datos menos sensibles se almacenan en la nube [68].

Sin embargo, el modelo de negocio de software híbrido requiere un equipo más grande de desarrolladores front-end que puedan manejar las tareas de interfaz y actualizar el software para garantizar que la interacción entre los servicios locales y los basados en la nube sea lo más integrada y fluida posible [68].

El modelo propuesto para la comercialización de TeamSoft+ adopta esta estrategia híbrida que sintetiza las ventajas más significativas de ambos paradigmas (SaaS y PaaS). Este enfoque puede conceptualizarse como "Producto con Gestión de Servicio". En esencia, el núcleo de la solución—el algoritmo de conformación de equipos y su interfaz de usuario—se distribuye como un producto tradicional instalable (un artefacto descargable que se ejecuta en la infraestructura del cliente). Sin embargo, todo el ciclo de vida comercial y de soporte que rodea a este producto se gestiona a través de un servicio centralizado en la nube.

Este modelo se inspira en actores consolidados del mercado. JetBrains, con sus entornos de desarrollo integrado (IDEs) como IntelliJ IDEA, es un ejemplo paradigmático: los desarrolladores pagan una suscripción que les da derecho a descargar y usar el software en su máquina local, mientras que la activación de la licencia, la gestión de la cuenta y la descarga de actualizaciones se canalizan a través de un portal web. De manera análoga, Microsoft con Windows o Adobe con Creative Cloud han migrado hacia modelos donde el software se instala localmente para un rendimiento óptimo, pero la licencia, facturación y servicios complementarios (como la nube de Adobe) son gestionados de forma remota. Para TeamSoft+, este modelo permite a los clientes finales (empresas, universidades) retener el control absoluto sobre sus datos sensibles de recursos humanos y operar el software en sus propios servidores, al mismo tiempo que establece para los desarrolladores una relación comercial directa, facilitando la distribución, el control de versiones y el soporte.

- **Componente Producto**

El Componente constituye el núcleo funcional de la solución y representa el foco principal del trabajo de desarrollo y refactorización descrito en esta tesis. Se trata de la aplicación TeamSoft+ propiamente dicha, que será descargada, instalada y ejecutada en los servidores. Este componente es fundamentalmente un software cliente-servidor autónomo que contiene toda la lógica de negocio, los algoritmos metaheurísticos (basados en la biblioteca Biciam) y la interfaz de usuario necesaria para que los usuarios finales formen y gestionen sus equipos.

- **Componente Servicio**

El Componente "Servicio" actúa como el sistema nervioso central del modelo de negocio y, si bien su desarrollo completo escapa al alcance inmediato de esta tesis, su diseño conceptual es una parte integral de la fundamentación. Esta plataforma web se erige como el punto único de contacto entre el proveedor (nosotros) y el cliente, encargándose de gestionar toda la relación comercial y de soporte. Sus funcionalidades principales se pueden desglosar en:

1. Portal de Autenticación y Gestión de Cuentas: Los clientes potenciales y existentes se registran en este portal, creando un perfil único en la plataforma.
2. Sistema de Gestión de Licencias: Este es el corazón comercial del servicio. Permite a los clientes adquirir suscripciones o licencias perpetuas y gestionar sus

métodos de pago. Cuando un usuario introduce una clave de licencia en el Componente "Producto" instalado localmente, este se comunica con este servicio para validar su autenticidad y estado (activa, expirada).

3. Centro de Descargas y Gestión de Versiones: La plataforma servirá como repositorio oficial y seguro desde donde los clientes con licencias activas podrán descargar la última versión estable del Componente "Producto", así como parches y actualizaciones menores. Esto asegura que todos los usuarios tengan acceso a las últimas mejoras y correcciones de seguridad.
4. Portal de Soporte y Documentación: Concentrará el conocimiento base, manuales de usuario, tutoriales y un sistema para asistencia técnica, mejorando la experiencia del cliente y reduciendo la carga de soporte directo.

1.5 Fundamentación de las tecnologías a utilizar para el desarrollo de la aplicación

Antes de exponer la fundamentación de la selección del lenguaje resulta importante precisar que esta decisión estuvo directamente ligada a la selección de Biciam [44] como biblioteca de clases que ofrece un conjunto de algoritmos metaheurísticos para la solución de problemas de optimización.

Lenguaje de programación

Dado que BICIAM está desarrollada en Java, se tomó la decisión de usar ese lenguaje para el desarrollo del backend de la herramienta, con el objetivo de simplificar la integración. Entre las ventajas de Java destacan [69, 70]:

- Es multiplataforma, esto significa que cuando se programa en Java, no es necesario conocer a priori el tipo de ordenador o el sistema operativo para el que se programa.
- No existen problemas con la liberación de memoria, ya que el programador no es el responsable de esa tarea, sólo necesita solicitar la memoria al sistema.
- Posee bibliotecas estándares para realizar multitud de operaciones comunes.

1.5.1 Framework back-end

Un framework back-end es un conjunto de herramientas, librerías y convenciones que facilitan el desarrollo de aplicaciones en el lado del servidor. Proporciona una estructura base para construir aplicaciones web, APIs, servicios o sistemas que manejen la lógica de negocio, la interacción con bases de datos, la autenticación, la seguridad y otras funcionalidades necesarias para el funcionamiento de una aplicación [71, 72].

Ofrecen una arquitectura predefinida (como MVC) para organizar el código de manera eficiente, permiten reutilizar componentes comunes, como conexiones a bases de datos, autenticación y manejo de solicitudes HTTP, incluyen medidas para proteger la aplicación contra vulnerabilidades comunes, como inyecciones SQL o ataques XSS, y proporcionan herramientas para interactuar con bases de datos de manera sencilla [71-73].

Entre los frameworks back-end más populares se encuentran Node.js, NestJS, Django, FastAPI, Spring Boot y Laravel [73].

Debido a que es necesario el acoplamiento con la biblioteca BICIAM y su lenguaje de desarrollo es Java, de todos los frameworks anteriormente mencionados Spring Boot es el framework de desarrollo de ese lenguaje:

Spring Boot es un framework de código abierto para el desarrollo de aplicaciones Java empresariales. Diseñado para facilitar el inicio rápido de los equipos de desarrollo, Spring Boot crea aplicaciones Spring independientes sin las opciones de configuración, a veces tediosas, que requiere la versión completa de Spring.

El objetivo del proyecto Spring es crear una experiencia mucho más rápida y optimizada mediante la configuración predictiva: utiliza algoritmos para seleccionar qué módulos y características de Spring habilitar, qué bibliotecas de terceros instalar y, posteriormente, configurarlas automáticamente. Además, añade numerosas funcionalidades no relacionadas con el código a un proyecto Java, como servidores web integrados, sistemas de seguridad, métricas y análisis [74, 75].

Spring Boot ofrece varias ventajas a los desarrolladores de Java sobre el uso de Spring Framework en sí. En términos generales, si los requisitos del proyecto pueden cumplirse con

Spring o Spring Boot, es mejor usar Spring Boot para aprovechar la configuración simplificada y el aumento de la productividad. Las ventajas específicas incluyen [75]:

- **Desarrollo acelerado.** Con Spring Boot, los equipos de desarrollo pueden empezar a trabajar de inmediato. Los algoritmos predefinidos de Spring Boot detectan, cargan y configuran los módulos, funciones y bibliotecas de terceros necesarios para el proyecto. En cambio, con Spring, los desarrolladores tienen que seleccionar manualmente qué incluir.
- **Gestión de dependencias.** Las dependencias se producen cuando una parte del código de una aplicación depende de la existencia y el correcto funcionamiento de bibliotecas, módulos, plugins, microservicios y API. Spring Boot incluye soporte integrado para la inyección de dependencias (DI), lo que permite incorporar objetos y funciones definidos externamente a la aplicación con un mínimo esfuerzo. Esto permite a los desarrolladores centrarse en la lógica de su programa en lugar de preocuparse por los detalles técnicos específicos de los recursos externos.
- **Capacidades listas para producción.** El objetivo no es escribir software, sino ejecutarlo. El empaquetado y la distribución de aplicaciones Java pueden ser complejos debido a la falta de estándares ampliamente aceptados para su empaquetado, y lo mismo ocurre con las aplicaciones que utilizan el framework Spring. Spring Boot resuelve este problema permitiendo a los equipos elegir entre archivos JAR ejecutables o archivos WAR desplegables. Las aplicaciones también se pueden desplegar directamente en contenedores o ejecutar como servicios de sistemas Windows o Unix/Linux.

1.5.2 Framework de front-end

Los frameworks de front-end son colecciones de código HTML, CSS y JavaScript estandarizado, escritas previamente, que proporcionan una estructura y funcionalidades predefinidas para facilitar el desarrollo de interfaces de usuario interactivas en aplicaciones web. Ellos permiten un desarrollo web más eficiente y estructurado, al optimizar el flujo de trabajo y la calidad del software. Suelen proporcionar un conjunto de herramientas y bibliotecas para crear interfaces de usuario, manejar eventos e interactuar con APIs. Ayudan a los desarrolladores a lograr coherencia entre proyectos, acelerar el desarrollo y reducir la complejidad del código [72, 76].

Entre los frameworks más populares y utilizados en la actualidad se encuentran: Vue.js, React.js y Angular.js. Estos frameworks se caracterizan por el uso de la reactividad, la cual facilita el desarrollo de aplicaciones web, ya que permiten que cualquier modificación en los datos se refleje automáticamente en la interfaz sin necesidad de intervención manual por parte del desarrollador. La presencia de esta característica es vital para la aplicación a desarrollar, donde se debe mostrar en tiempo real, respuestas del sistema al usuario [76-79].

Se decide seleccionar Vue.js para la implementación del nuevo front-end de la herramienta. Vue.js es un framework de JavaScript liviano, creado por Evan You. La popularidad de Vue.js se ha incrementado en los últimos años, dado por su baja curva de aprendizaje y alto rendimiento en aplicaciones relativamente sencillas, considerado superior al de las otras dos tecnologías mencionadas. Otro elemento a considerar en la decisión es el conocimiento previo de este framework por parte del equipo de trabajo [80].

La decisión de migrar desde Java Server Faces (JSF), tecnología utilizada en la versión anterior de TeamSoft+, hacia Vue.js se fundamenta en la necesidad de superar varias limitaciones inherentes al modelo de desarrollo anterior. JSF, al ser un framework de tipo *server-side* [81], genera una fuerte dependencia y acoplamiento con el backend, lo que resulta en una arquitectura monolítica donde la lógica de presentación y la de negocio se entrelazan. Esto dificulta el desarrollo, testeo y mantenimiento independiente de la interfaz de usuario. Por el contrario, Vue.js, como framework *client-side* moderno, permite la creación de una aplicación de una sola página (SPA) que se comunica con el backend a través de APIs RESTful [80], estableciendo una clara separación de responsabilidades. Esta arquitectura desacoplada mejora radicalmente la mantenibilidad y escalabilidad del front-end. Además, Vue.js ofrece un mejor rendimiento en el cliente al minimizar las recargas completas de página y manejar las actualizaciones de la interfaz de manera reactiva y eficiente. Finalmente, la experiencia de desarrollo (DX) es significativamente superior, con un ecosistema de herramientas (como Vue CLI, Vue DevTools y Vite) que agilizan la creación de componentes reutilizables, la gestión de estado y la detección de errores, aspectos que se encontraban notablemente más rígidos y eran propensos a errores en el ciclo de desarrollo con JSF.

1.5.3 Gestor de base de datos

Se decidió usar el gestor de bases de datos PostgreSQL [82] tomando en cuenta:

- Su capacidad para manejar grandes volúmenes de datos.
- La necesidad de garantizar la consistencia de la base de datos a partir del hecho de que PostgreSQL mantiene la integridad referencial entre las llaves primarias y foráneas, y la eliminación y actualización en cascada.
- Da la posibilidad de realizar una subconsulta en una consulta SQL.
- Presenta llaves primarias y la posibilidad de que campos de una tabla sean únicos. En el caso de las llaves foráneas, permite las operaciones de borrar o modificar en cascada a partir de ellas.

1.5.4 Herramientas de mapeo relacional

El proceso de mapear objetos Java a las tablas de una base de datos relacional y viceversa se denomina "Mapeo Objeto-Relacional" (ORM). Para abordar este desafío de forma estandarizada en el ecosistema Java, se creó la especificación **Java Persistence API (JPA)**. JPA define un conjunto de anotaciones, interfaces y reglas que permiten a los desarrolladores gestionar datos relacionales utilizando objetos Java de forma declarativa, trabajando predominantemente con objetos en lugar de con sentencias SQL nativas, lo que agiliza el desarrollo y reduce el código boilerplate [83-85].

Si bien JPA es una especificación que cuenta con varias implementaciones, como EclipseLink, Apache OpenJPA e Hibernate, este último se ha consolidado como el proveedor de persistencia más popular y ampliamente adoptado en la industria. Hibernate no solo es la implementación de referencia *de facto*, sino que actúa como un motor de persistencia robusto y de alto rendimiento que cumple con el estándar JPA, a la vez que ofrece características adicionales y una extensa comunidad de soporte [83, 86-88].

Una de las principales ventajas de utilizar una especificación como JPA es la portabilidad del código. Al desarrollar contra la API estándar de JPA, el código de persistencia se mantiene independiente del proveedor subyacente. En teoría, esto permite cambiar de implementación (por ejemplo, de Hibernate a EclipseLink) con modificaciones mínimas en el código, generalmente limitadas a la configuración. Sin embargo, en la práctica, Hibernate se ha convertido en la elección predominante debido a su madurez, optimizaciones avanzadas de rendimiento (como el cache de primer y segundo nivel) y su rico ecosistema de herramientas.

Para el desarrollo de TeamSoft+, se seleccionó Hibernate como el proveedor de JPA para garantizar una solución de persistencia eficiente, bien documentada y alineada con las mejores prácticas más comunes en el desarrollo de aplicaciones Java empresariales modernas.

1.5.5 Spring Security

Spring Security [89] provee un conjunto de servicios de seguridad para software de aplicaciones empresariales basadas en J2EE. En Spring Security están previstas las dos partes fundamentales de la seguridad: la autenticación y la autorización.

La capacidad de Spring Security de proporcionar los elementos de seguridad que proponen los estándares, con un mínimo de codificación, con alto grado de confiabilidad y con una fácil configuración de todos sus elementos, hizo que se decidiera adoptar esta plataforma para soportar la seguridad del sistema [74].

1.5.6 SonarQube

SonarQube es una plataforma de código abierto diseñada para analizar y mejorar la calidad del código fuente en proyectos de software. Utiliza análisis estático para detectar errores, vulnerabilidades y problemas de seguridad, promoviendo buenas prácticas de desarrollo [90-92].

Ventajas de utilizar SonarQube:

- SonarQube identifica problemas como errores comunes y vulnerabilidades de seguridad antes de que el código entre en producción.
- Detecta bloques repetidos o similares de código, fomentando un diseño más limpio y promoviendo la reutilización eficiente del código.

1.6 Conclusiones parciales

Una vez culminado el capítulo se arriban a las siguientes conclusiones:

1. La formación de equipos se considera un factor crítico en el éxito de las organizaciones. Es un proceso complejo que debe considerar múltiples factores tanto individuales como colectivos.

2. El modelo de formación de múltiples equipos que sirve como base para este trabajo logra integrar factores tanto intrínsecos como de extrínsecos, dado que incluye funciones objetivo y restricciones asociadas a cada uno de estos factores.
3. Resulta conveniente utilizar Biciam en la herramienta Teamsoft+ ya que es una de las bibliotecas más completas, pues permite la solución de problemas multiobjetivo e incorpora variantes de algoritmos multiobjetivo de trayectoria y poblacionales.
4. A pesar de las potencialidades de Teamsoft+, resulta necesario enriquecer el modelo mediante la incorporación de funciones objetivos que permitan tomar en cuenta factores demográficos como edad y religión cruciales para fomentar la diversidad y la armonía dentro de los equipos.
5. La existencia de una elevada deuda técnica en la base de código actual de TeamSoft+, manifestada en "malos olores" de código y una arquitectura monolítica, justifica plenamente la necesidad de un proceso de refactorización profunda. Este proceso es un prerequisite indispensable no solo para mejorar la mantenibilidad y escalabilidad del software, sino también para habilitar la transición hacia el modelo de negocio híbrido definido, el cual combina las ventajas de un software instalable y de alto rendimiento con la practicidad comercial y de distribución de una plataforma de servicio centralizada.
6. Se identificaron las siguientes tecnologías para el desarrollo de la nueva versión de TeamSoft+: al lenguaje Java para el backend, el framework Vue.js para el frontend (sustituyendo a JSF), Hibernate como proveedor de JPA para el mapeo objeto-relacional, el gestor de base de datos PostgreSQL, y otras herramientas de soporte como Spring Security para la seguridad y SonarQube para la garantía de calidad del código.

Capítulo 2: Solución propuesta

2.1 Introducción

El presente capítulo tiene como objetivo presentar el diseño y desarrollo de la solución propuesta para abordar las insuficiencias identificadas en la versión actual de TeamSoft+. En primer lugar, se enriquece la aplicabilidad de la incorporando los factores demográficos "religión" y "grupo de edades", lo que implica el diseño de nuevas funciones objetivo y la modificación del esquema de base de datos. En segundo lugar, y como pilar fundamental para la escalabilidad futura, se lleva a cabo un rediseño arquitectural, migrando de una aplicación

monolítica a una arquitectura basada en una API REST, lo que permite la separación de la lógica de presentación y la lógica de negocio. Este capítulo detalla este proceso, describiendo la implementación de la API, las técnicas de refactorización aplicadas para mejorar la calidad estática del código y los principios de diseño que guiarán la nueva interfaz de usuario, sentando las bases técnicas para la culminación del proyecto en su etapa de tesis.

2.2 Incorporación de nuevas funciones objetivo

La evolución de TeamSoft+ requiere dotarla de la capacidad para gestionar factores demográficos críticos, identificados en la literatura como generadores potenciales de "líneas de falla" dentro de los equipos. La incorporación de la "religión" y el "grupo de edades" permite a los decisores formar equipos no solo con base en competencias y habilidades, sino también en criterios de diversidad, homogeneidad o balance demográfico, fomentando así una mayor armonía y cohesión grupal [93].

2.2.1 Funciones objetivo asociadas al factor religión y grupo de edad

Para integrar los nuevos factores demográficos en el modelo de optimización multiobjetivo de TeamSoft+, se diseñaron tres tipos de funciones objetivo para cada factor (religión y grupo de edades), permitiendo al usuario seleccionar el criterio deseado durante la configuración del proceso de formación de equipos:

1. **Homogeneidad:** Esta función objetivo busca maximizar la similitud interna dentro de cada equipo con respecto al factor. Por ejemplo, formar equipos donde la mayoría de sus miembros profesen la misma religión o pertenezcan al mismo grupo etario.
2. **Heterogeneidad:** En contraposición a la anterior, esta función busca maximizar la diversidad dentro de cada equipo. El objetivo es crear equipos con una mezcla equilibrada de diferentes religiones o rangos de edad.
3. **Balance:** Esta función objetivo opera a un nivel superior, buscando equilibrar la composición demográfica entre todos los equipos que se están formando simultáneamente. El objetivo es evitar que un equipo concentre a todos los miembros de una determinada religión o grupo de edad, distribuyéndolos de manera justa entre todos los equipos.

En la Figura 2 se observan el diagrama de clases de la biblioteca de clases BICIAM, en color azul celeste se encuentran marcadas las clases añadidas durante el desarrollo de las nuevas funciones objetivo.



Para soportar las nuevas funcionalidades, fue necesario realizar modificaciones en el modelo de base de datos de TeamSoft+. Estas modificaciones aseguran la persistencia de la información relativa a los nuevos factores y su vinculación con las personas. Los cambios principales se reflejan en el siguiente diagrama y en su explicación:

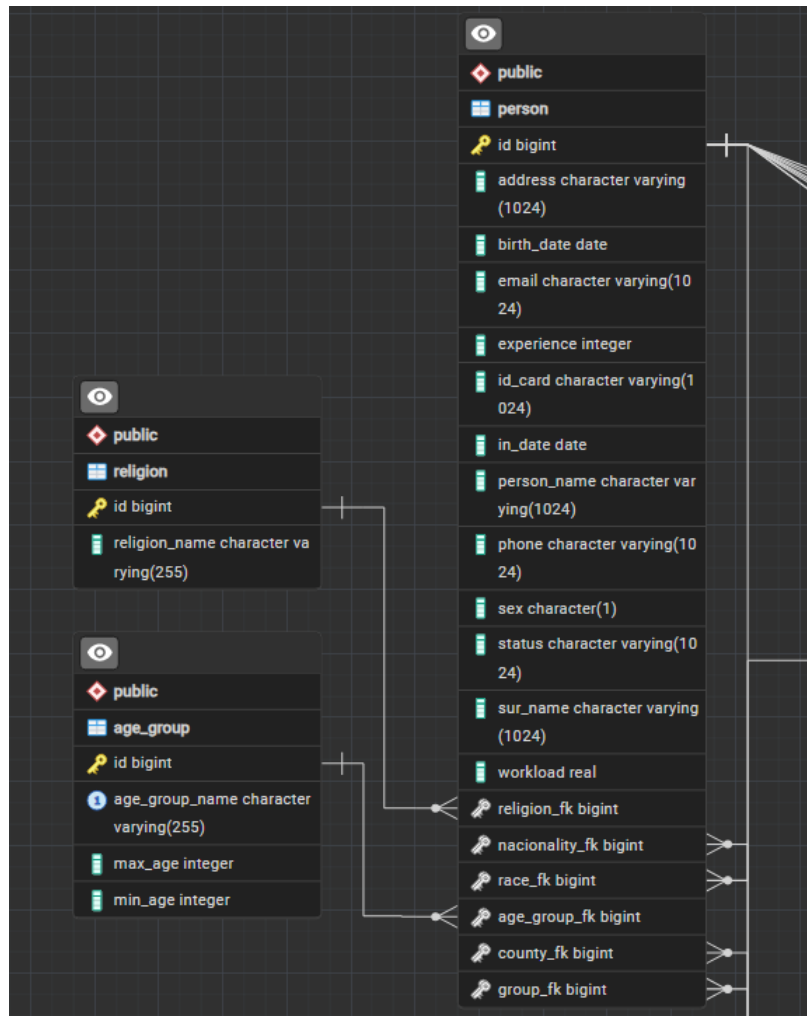


Figura 3: Modificaciones en la base de datos.

Las modificaciones implementadas fueron:

1. **Creación de la tabla religion:** Esta nueva tabla actúa como un nomenclador, almacenando los diferentes credos religiosos que pueden asignarse a las personas (por ejemplo: católico, protestante, budista, etc.). Contiene campos básicos como id (llave primaria) y nombre.
2. **Creación de la tabla age_group:** Similar a la anterior, esta tabla funciona como un nomenclador para definir los rangos de edad (por ejemplo: 18-25, 26-35, 36-45, etc.), con su respectivo id y nombre.

3. **Modificación de la tabla persona:** Se agregaron dos nuevas llaves foráneas y una columna `birth_date` a esta tabla central:

- `religion_fk`: que referencia a la tabla `religion`.
- `grupo_edad_fk`: que referencia a la tabla `age_group`.
- `birth_date`: que hace referencia a la fecha de nacimiento de la persona, es un dato autocalculable.

Estas relaciones permiten asociar a cada persona con una religión y un grupo de edad específicos. La integridad referencial garantiza la consistencia de los datos, y el diseño facilita la gestión de estos nomencladores a través de la interfaz administrativa, haciendo el sistema más aplicable a múltiples contextos y preparado para el cálculo de las nuevas funciones objetivo durante el proceso de formación de equipos.

2.3 Rediseño de la arquitectura

La herramienta en su estado actual sigue una arquitectura monolítica de modelo vista controlador (MVC), donde la lógica de presentación, la lógica de negocio y el acceso a datos estaban fuertemente acoplados. Este acoplamiento dificultaba el mantenimiento, la incorporación de nuevas funcionalidades y, fundamentalmente, imposibilitaba la escalabilidad necesaria para ofrecer el software bajo el modelo híbrido definido.

Para superar estas limitaciones, se ejecutó un rediseño arquitectural completo, migrando hacia una Arquitectura Cliente-Servidor. Esta nueva arquitectura establece una clara separación de responsabilidades entre el cliente (*front-end*) y el servidor (*back-end*), comunicándose a través de una API RESTful [94].

Entre sus ventajas con respecto al proyecto en cuestión se encuentran [95]:

1. Los clientes no necesitan conocer detalles específicos del sistema (software o hardware), aspecto fundamental para solucionar la problemática.
2. Los clientes suelen estar en computadoras personales, mientras que el servidor está en una máquina más potente, que se encargará de almacenar la información centralizada en una sola base de datos, lo cual es requerido para realizar estudios y, análisis científicos y académicos sobre los resultados de los test psicológicos.

El *back-end* se desarrolla como una aplicación independiente que encapsula toda la lógica de negocio y la persistencia de datos, mientras que el *front-end* se plantea como una aplicación web que consume dicha API. Este desacoplamiento no solo mejora radicalmente la mantenibilidad y permite que ambos componentes evolucionen de forma independiente, sino que también constituye el pilar tecnológico que habilita la futura comercialización de TeamSoft+ como un producto instalable con gestión centralizada de licencias y actualizaciones.

El sistema se dividirá en múltiples capas (o niveles) independientes, cada una con responsabilidades y funciones específicas. Esta separación en capas permite una mejor organización, modularidad y escalabilidad del sistema. Cada capa es independiente de las demás, lo que facilita el desarrollo, mantenimiento y actualización del sistema [96].

Las capas generalmente pueden reutilizarse en otros sistemas, lo que reduce el tiempo de desarrollo. Se pueden agregar o eliminar las mismas fácilmente para satisfacer las necesidades cambiantes del sistema. Cada capa se encarga de una función específica, lo que mejora la legibilidad y el mantenimiento del código. Las diferentes capas pueden usar diferentes tecnologías, lo que permite flexibilidad y adaptación [96].

El sistema, como se observa en la Figura 2, se divide en varias capas definidas por reutilización, con el objetivo de al escalar el sistema con acciones como añadir nuevos factores y funcionalidades a la aplicación, la arquitectura facilite dichas acciones y permita mantener una consistencia en la estructura del proyecto. En fondo rojo se señalan los paquetes en los cuales se centra el trabajo desarrollado:

- **Capa específica:** Constituye la capa del sistema que contiene elementos no reutilizables y que se relacionan fuertemente con las reglas del negocio de la aplicación. En esta capa se encuentran:
 - Interfaces gráficas de TeamSoft+: Establece la forma de interacción con el usuario, la recepción y validación de los datos. Aquí se encuentran las interfaces gráficas de la aplicación pues son los elementos más específicos del sistema.
- **Capa general:** Llevan a cabo la lógica del negocio que puede ser reutilizada en otros proyectos. Maneja la comunicación entre las aplicaciones *front-end* y *back-end*, e interactúa con la información en base de datos. En esta capa se encuentran:

- Módulos *Front-End*: Agrupa la lógica de conexión con la API, así como las funcionalidades relacionadas con el almacenamiento del token de autorización, componentes visuales reutilizables, funciones útiles para agilizar el desarrollo, entre otros elementos que ya se han desarrollado en otros proyectos y que pueden ser reciclados en el nuevo *front-end*.
- Módulos *Back-End*: Agrupa la lógica de negocio de cada módulo y sus conexiones a la base de datos. Dentro de los módulos se encuentran los controladores (encargados de procesar las peticiones), los servicios (contienen la lógica de negocio) y las entidades (representaciones de las entidades de la aplicación).
- **Capa intermedia:** Establece las bibliotecas, *frameworks* y software que requieren las capas superiores para funcionar. Agrupa el *framework* de *front-end* Vue.js, el cual consume del *framework* Primevue (biblioteca de componentes), y la biblioteca Axios, encargada de realizar las peticiones. Por otra parte, se encuentra Sringboot, que consume de Spring Framework, y de Spring Data JPA para poder realizar las consultas a la base de datos, que se encuentra desarrollada en PostgreSQL.
- **Capa Software del sistema:** Establece los elementos que utiliza la capa intermedia para poder realizar acciones como la comunicación, para la cual se utilizan los protocolos HTTP y TCP/IP.

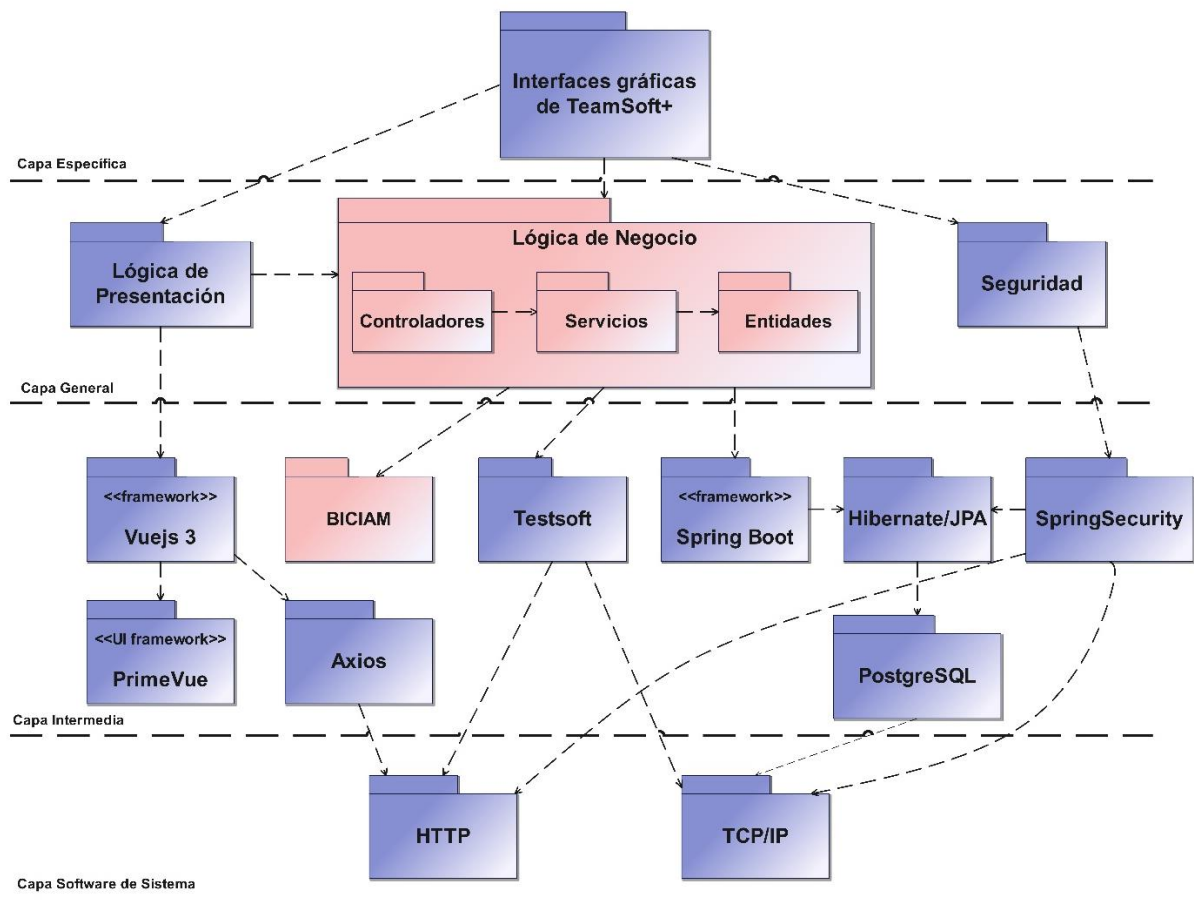


Figura 4: Representación del patrón N-Capas con un enfoque basado en la reutilización

2.4 Implementación de la API REST

La implementación de la API REST constituye el núcleo del trabajo de refactorización y modernización del *back-end*. Para garantizar un código limpio y mantenible se adoptó una estructura en capas y un conjunto de patrones de diseño específicos.

2.4.1 Diseño de los *endpoints* REST

El diseño de los *endpoints* sigue los principios y convenciones REST, priorizando la claridad, la predictibilidad y el uso correcto de los verbos HTTP. Para el módulo de nomencladores, que incluye las 17 entidades base del sistema (como Religión, Grupo de Edad, Competencias, Roles, etc.) se diseñaron 58 servicios en total y se estableció un esquema de URL uniforme. En la Figura 5 se muestran los *endpoints* generados para el controlador de religión donde

actualmente se tienen 5 *endpoints* creados, allí podemos ver el método HTTP del servicio y la URL.

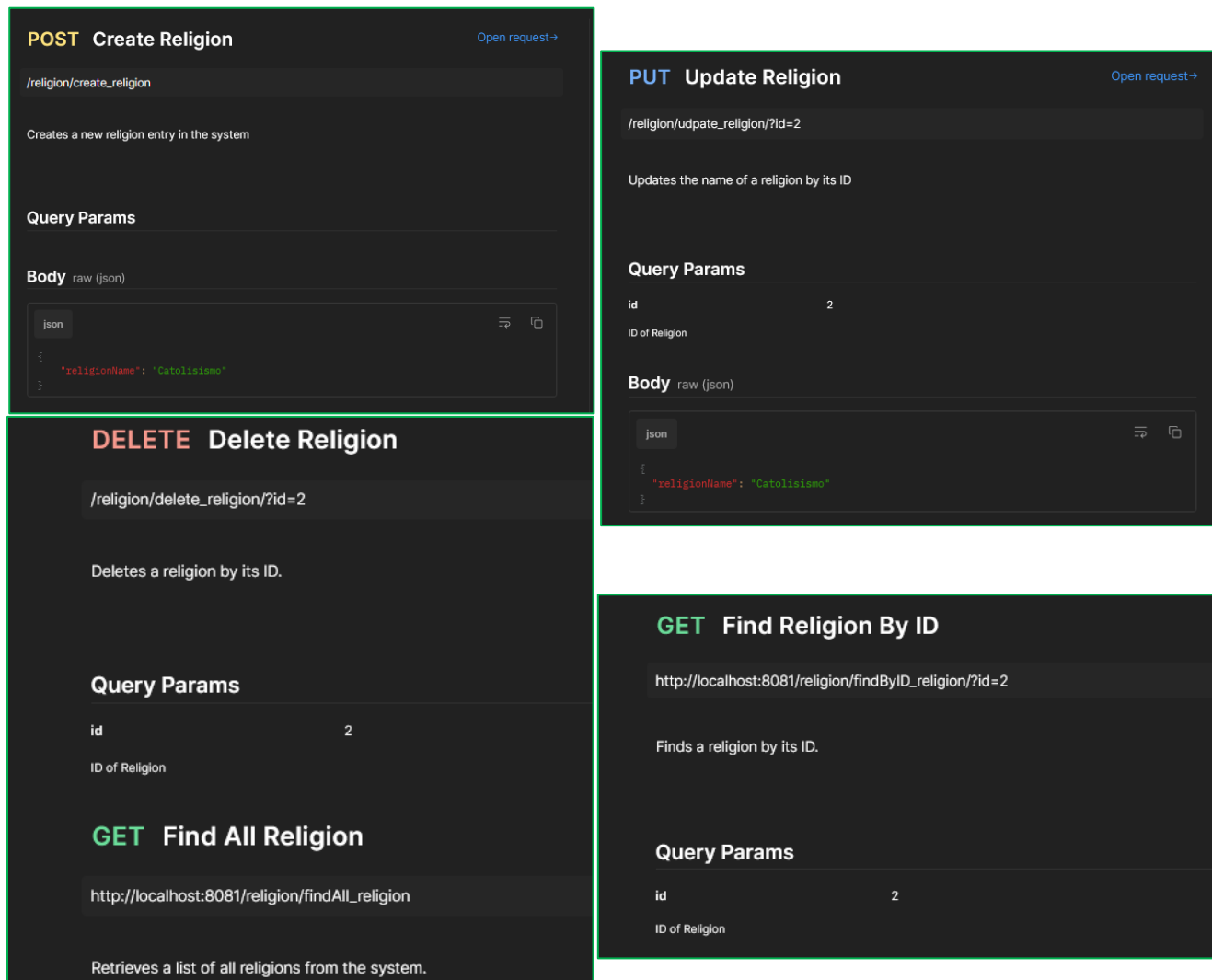


Figura 5: Postman UI - Grupo de controladores para la gestión de Religiones

Alguna de las buenas prácticas recomendadas (convenciones) [97, 98] que podemos ver que estén aplicadas en la Figura 5 son:

1. Uso de sustantivos para representar recursos
2. El recurso puede ser una colección o un singleton: `/api/v1/religion` representa una colección de religiones, mientras que `api/vi/religion/{id}` representa una religión en específico.
3. Uso de la barra inclinada `/` para indicar relaciones de jerarquía.

4. No utiliza la barra inclinada / al final de los recursos.
5. Nombramiento de toda la URL en minúscula.
6. Uso de guiones para mejorar la legibilidad.
7. No se hace uso de URLs para indicar funciones CRUD, para esto se utilizan los métodos HTTP.

Esta misma estructura se replica para el resto de los nomencladores, garantizando una API coherente y fácil de consumir por el futuro *front-end*. Las respuestas utilizan códigos de estado HTTP estándar (200 para éxito, 201 para creación, 400 para errores de validación, 404 para no encontrado) y el formato JSON para el intercambio de datos.

Las imágenes siguientes muestran ejemplos de peticiones a los diferentes controladores, el tipo de petición y los distintos estados HTTP de las respuestas.

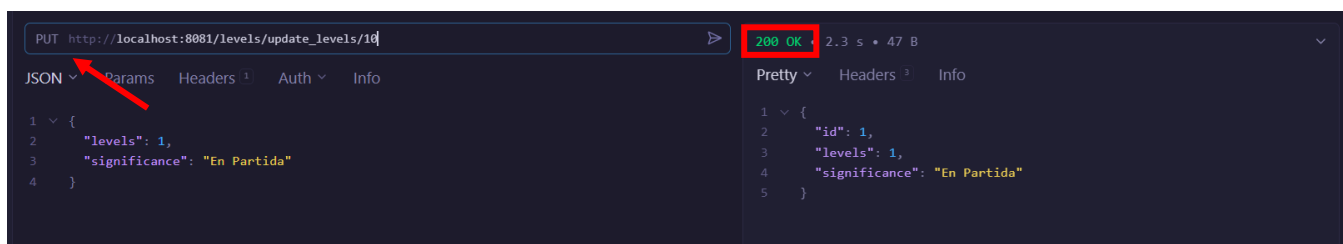


Figura 6: Ejemplo de petición exitosa al actualizar un nivel de competencia.

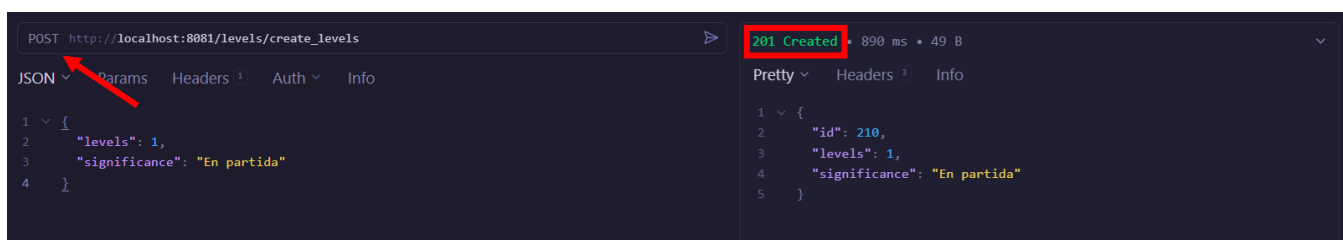


Figura 7: Ejemplo de petición exitosa al crear un nivel de competencia.

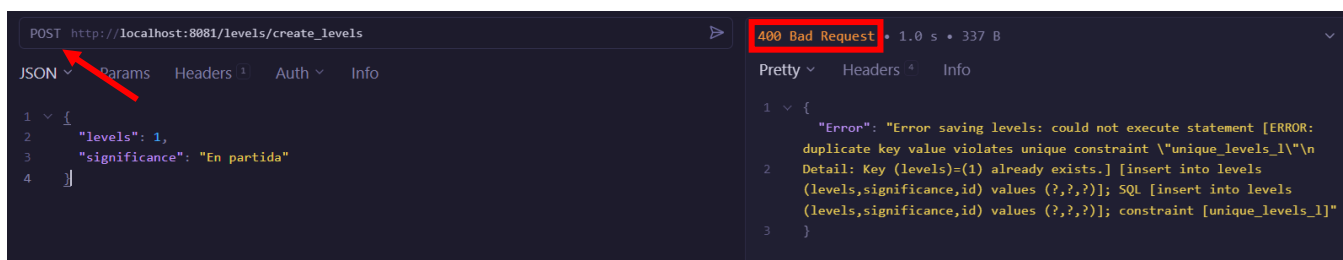


Figura 8: Ejemplo de petición errónea al crear un nivel de competencia.

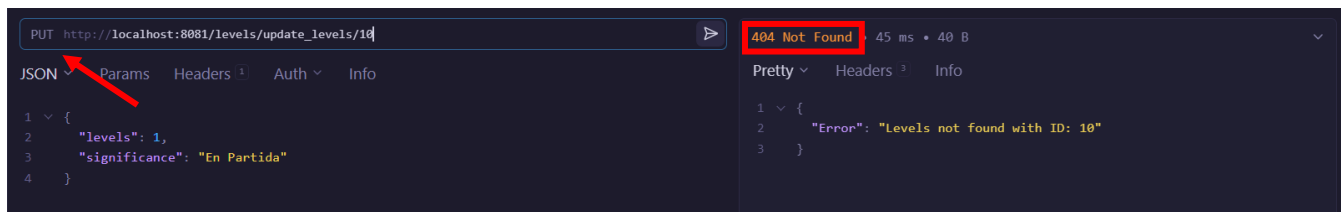


Figura 9: Ejemplo de petición para un nivel de competencia que no se encuentra en la base de datos.

2.4.2 Estructura modular

La API se organiza en una arquitectura por módulos bien definidos, que separa las responsabilidades y facilita el *testing* y el mantenimiento. La estructura del proyecto *back-end* se divide en los siguientes módulos o capas:

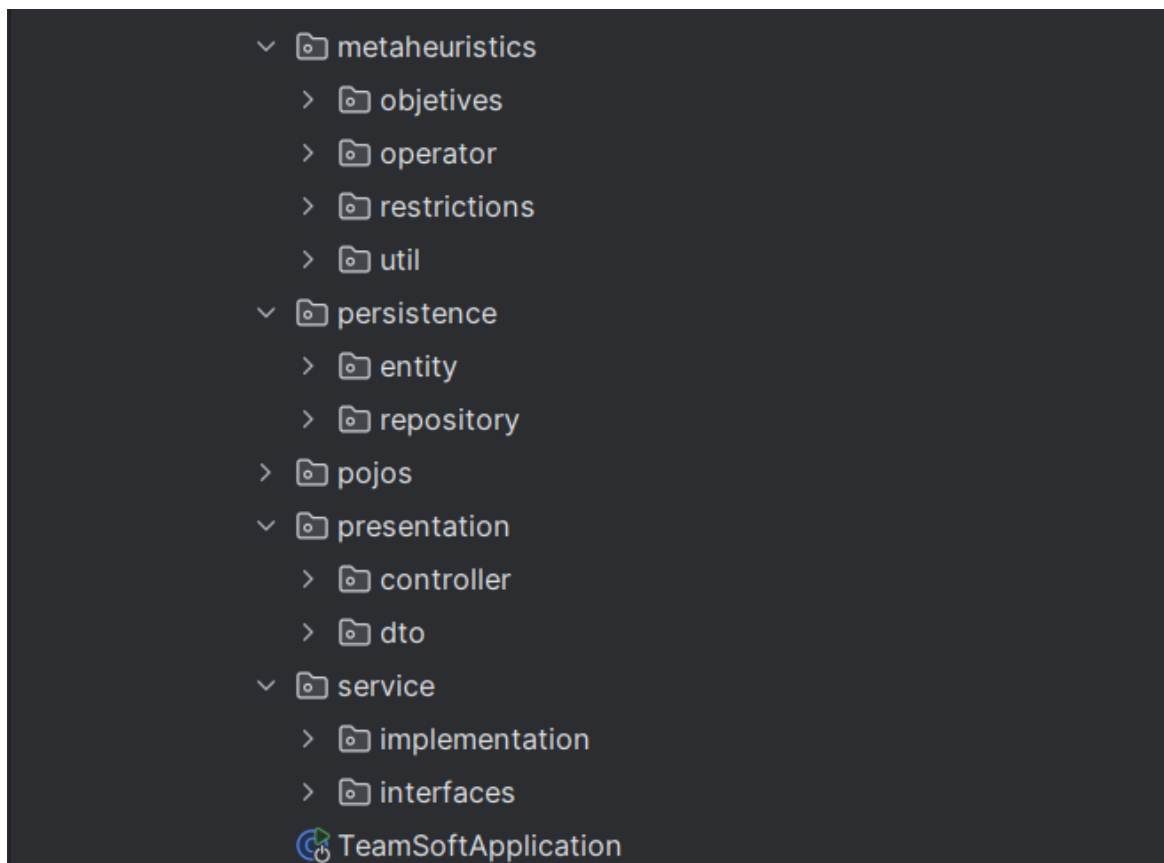


Figura 10: Estructura modular de la nueva versión de TeamSoft+.

1. Módulo de Persistencia (persistence):

1. Contiene las clases anotadas con JPA (`@Entity`) que representan las tablas de la base de datos (por ejemplo: `ReligionEntity.java`, `AgeGroupEntity.java`).
2. Contiene las interfaces que extienden de `JpaRepository` de Spring Data JPA (por ejemplo: `iReligionRepository.java`). Esta capa se encarga de abstraer todas las operaciones de acceso y consulta a la base de datos.
2. **Módulo de Servicio (service):** Contiene las interfaces y sus implementaciones (por ejemplo: `ReligionService.java` y `ReligionServiceImpl.java`). Aquí reside la lógica de negocio y actúa como un intermediario entre los controladores y los repositorios.
3. **Módulo de Presentación (presentation):** Contiene las clases anotadas con `@RestController` (por ejemplo: `ReligionController.java`). Su única responsabilidad es manejar las solicitudes HTTP, invocar los servicios correspondientes y devolver las respuestas adecuadas al cliente. También contiene los objetos de transferencia de datos (Data Transfer Objects, DTOs) que se utilizan para definir el formato de entrada y salida de la API, garantizando que no se exponga el modelo de persistencia interno directamente.

2.4.3 Patrones de diseño

Para resolver ciertas problemáticas se utilizaron un conjunto de patrones de diseño, los cuales, además de brindar una solución a las mismas, mejoran la calidad del código y su escalabilidad.

Inyección de Dependencias (DI):

Spring Framework gestiona automáticamente las dependencias entre las clases (por ejemplo, inyectando una instancia de `ReligionService` en `ReligionController`). Esto hace el código más testeable y flexible.

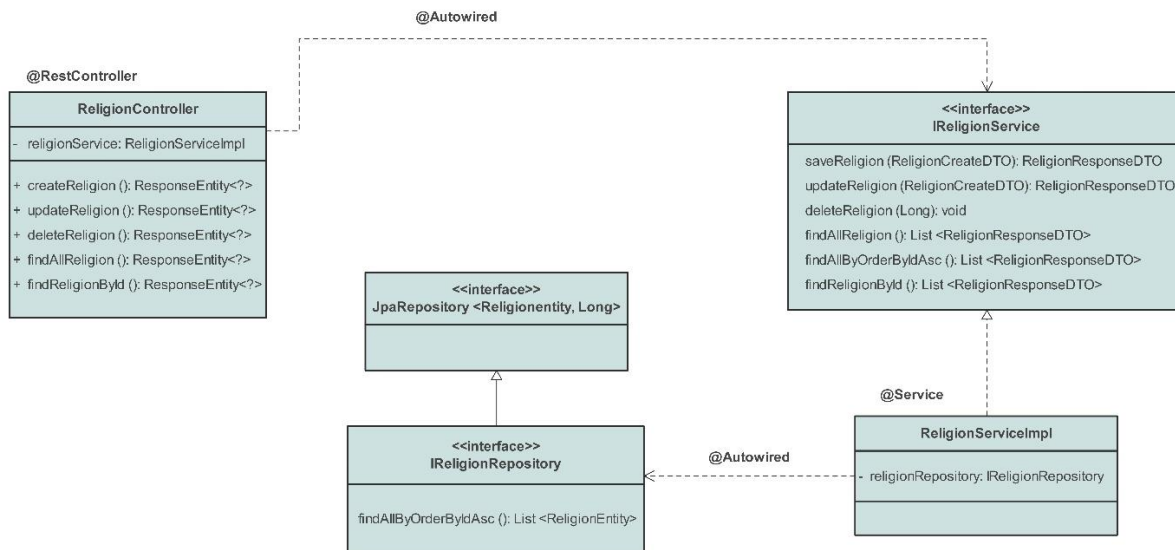


Figura 11: Representación del patrón Inyección de Dependencia en el proyecto.

Data Transfer Object (DTO):

Si bien en una primera etapa se pueden exponer las entidades directamente, se ha diseñado para usar DTOs en el futuro. Este patrón permite controlar exactamente qué datos se exponen en la API, evitando la exposición accidental de información sensible o la recursión infinita en relaciones bidireccionales.

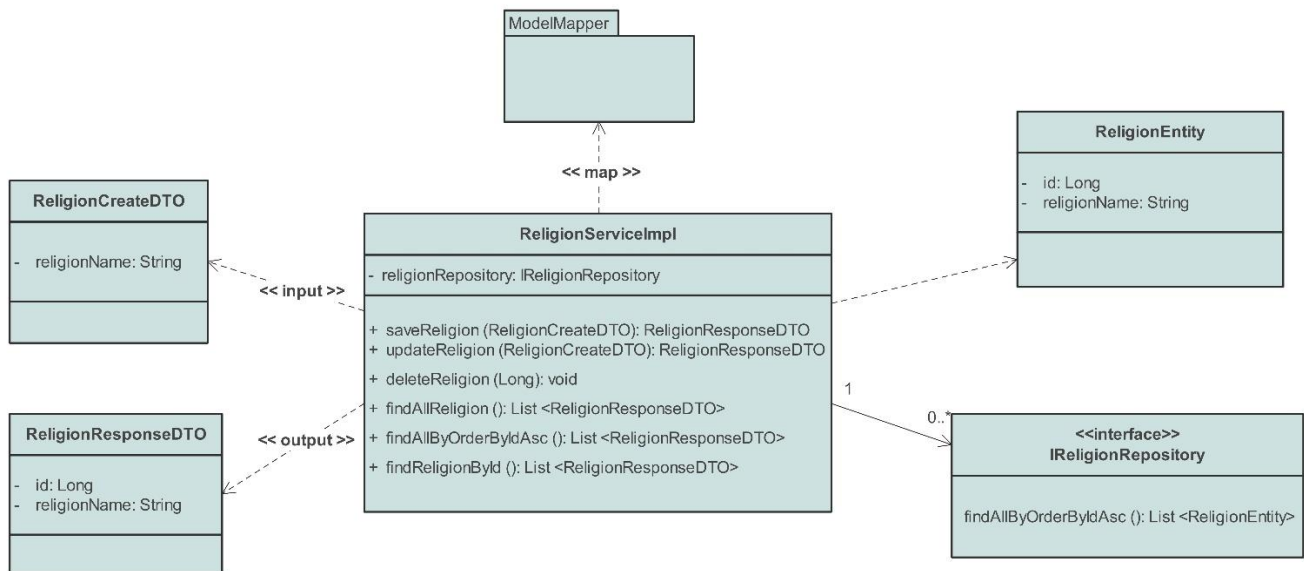


Figura 12: Representación del patrón DTO en el proyecto (Diagrama UML).

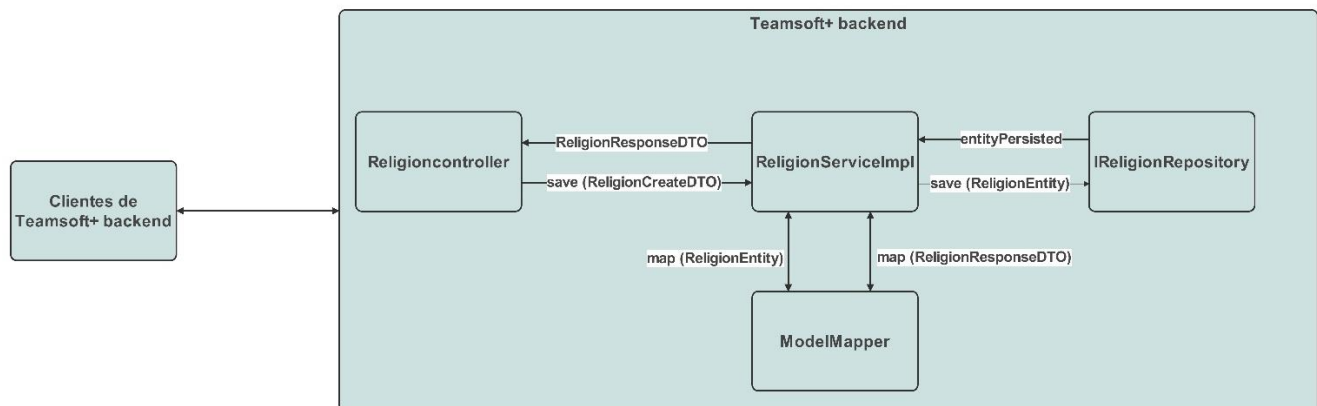


Figura 13: Representación del patrón DTO en el proyecto (flujo de mapeo y persistencia).

Patrón Singleton:

La intención de este patrón es garantizar que solo se cargue una instancia de una clase y que la instancia proporcione un punto de acceso global. Este patrón se evidencia al inicializar los parámetros que se van a tener en cuenta para la formación de los equipos, específicamente en el método initializeParameters de la clase TF_StepThreeController, controlador de la vista que

forma los equipos y en donde la instancia parametersTeam de la clase TeamFormationParameters se crea si la misma no está creada. Posteriormente, se guardan los parámetros seleccionados y se retornan en la instancia parametersTeam.

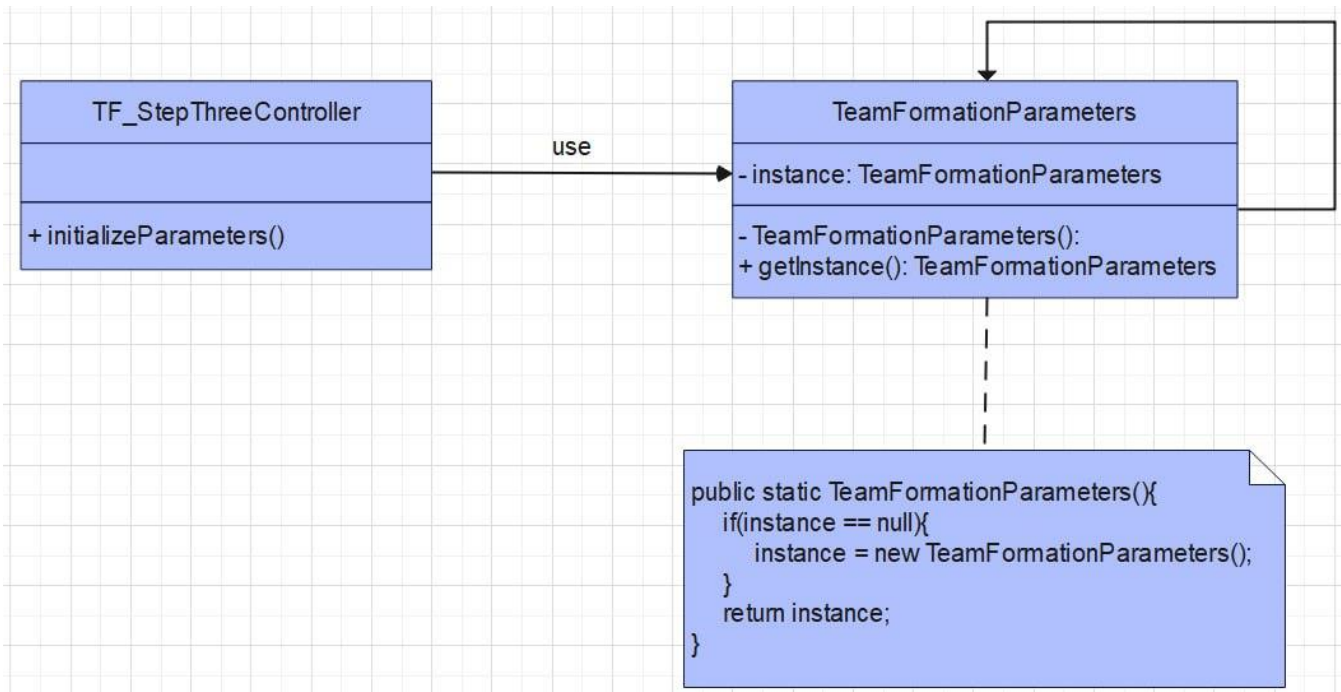


Figura 14: Representación del patrón Singleton en el proyecto.

La combinación de esta estructura en capas y estos patrones de diseño ha resultado en una base de código para la API que es modular, escalable y fácil de extender con nuevas funcionalidades en las próximas etapas del proyecto.

2.5 Conclusiones parciales

Una vez culminado el capítulo se arriban a las siguientes conclusiones:

1. Se logró enriquecer sustancialmente el modelo de formación de equipos de TeamSoft+ mediante la incorporación de los factores demográficos "religión" y "grupo de edades", diseñando tres tipos de funciones objetivo (homogeneidad, heterogeneidad y balance)

para cada uno, lo que permite a los decisores configurar criterios de diversidad más completos y adaptados a la dinámica organizacional contemporánea.

2. El rediseño arquitectural, migrando de una aplicación monolítica a una arquitectura basada en una API RESTful, estableció una separación entre la lógica de presentación y la lógica de negocio, fundamental para mejorar la mantenibilidad, facilitar el desarrollo independiente del *front-end* y el *back-end* y sentar las bases que permitan la transición hacia un modelo de negocio híbrido (SaaS/PaaS) escalable.
3. La implementación de la API REST se realizó siguiendo principios y patrones de diseño resultando en un código *back-end* más limpio, testeable y preparado para la incorporación futura de nuevas funcionalidades.

Capítulo 3:

3.1 Introducción

El presente capítulo tiene como objetivo validar la solución desarrollada mediante un conjunto de pruebas que permitan verificar el correcto funcionamiento del *back-end* desarrollado. se ejecutan un conjunto de pruebas que permiten verificar el cálculo correcto de los coeficientes de las funciones objetivo. Se presentan los resultados de la refactorización realizada y su impacto en la calidad estática del código, evaluada mediante la herramienta SonarQube. Asimismo, se diseñan y ejecutan pruebas funcionales para los *endpoints* de la API REST, utilizando tablas de casos de uso que aseguren el cumplimiento de los requisitos. Finalmente, se realizan pruebas de eficiencia para medir los tiempos de respuesta y el rendimiento general del sistema, garantizando que la nueva versión de TeamSoft+ mejora en tiempo de respuesta comparada con la versión anterior.

3.2 Diseño y Ejecución de pruebas para validar el cálculo de los coeficientes de las funciones objetivo implementadas

Para validar que se calculan de forma correcta los coeficientes de las funciones objetivo se utilizaron datos de estudiantes de primer año de la carrera de Ingeniería Informática de la CUJAE, a los cuales se le asignaron diferentes religiones (en algunos casos ficticias) con sus respectivos grupos de edades. En el Anexo 4 se muestran los datos de los estudiantes: nombre y apellido, religión y grupo de edad.

La estrategia utilizada para comprobar que se calcula de forma correcta los coeficientes de las funciones objetivo fue correr la herramienta para que forme los equipos tomando en cuenta los factores Religión y Grupo de Edad. Una vez que se cuenta con la propuesta de equipo que brinda TeamSoft+, se calcula a mano el coeficiente de la función objetivo y se chequea que coincide con el que calculó la herramienta. No se puede elaborar un caso de prueba clásico ya que como los algoritmos son combinatorios no se conoce las personas que pueden ser asignadas al equipo.

Todos los coeficientes deben normalizarse a valores entre 0 y 1, para ello se debe calcular el valor máximo y mínimo que puede tomar el coeficiente de la función objetivo.

(PONER TABLA CON LOS DATOS DE LAS PERSONAS)

3.2.1 Comprobar el funcionamiento de las funciones objetivo relacionadas con el factor Religión

Para calcular el coeficiente normalizado para las funciones objetivo relacionadas con el factor Religión se emplea la siguiente fórmula:

Coeficiente normalizado = valor / cantidad de equipos

valor = (cantidad de religiones diferentes en el equipo – valor min) / (valor máx – valor min)

El valor mínimo es 1 ya que en el peor de los casos al menos hay una religión en el equipo.

El valor máximo es la cantidad de religiones diferentes en la organización si la cantidad de personas del equipo es mayor o igual a la cantidad de nacionalidades diferentes en la organización, sino el valor máximo es la cantidad de personas del equipo.

Caso de prueba 1: Formación de equipo heterogéneo con factor Religión

Objetivo: Comprobar que se calcula correctamente el coeficiente de la función objetivo relacionada con el factor Religión y que se forma un equipo heterogéneo.

Como se muestra en la Figura 15 la herramienta formó un equipo heterogéneo, ya que está compuesto por diez personas de múltiples religiones, la cantidad de religiones en el equipo es 7:

Alberto Rodríguez Castro (Testigo de Jehová), Hans Concepción Álvarez (Judaísmo), Alejandro Aguirre García (Católico), Daniel Alejandro Corría Montanés (Católico), Eva Ercilia, Vázquez García (Ateo), Andy Daniel Gonzales Guevara (Yoruba), Alfredo Hernández Rodríguez (Yoruba), Gisselle Blanco Almeida (Bautista), Miguel Ángel López García (Agnóstico), Adriana Pérez Serafín (Ateo).



Figura 15: Propuesta de equipo heterogéneo (Factor Religión).

El valor mínimo de religiones diferentes es 1.

El valor máximo es 8 ya que la cantidad de personas del equipo es mayor que la cantidad de religiones diferentes que existe en el grupo (8 religiones < 10 personas en el equipo).

valor = (cant de religiones diferentes de cada equipo – valor mín) / (valor máx – valor mín)

valor = (7 - 1) / (8 - 1) = 6 / 7 = 0.8571

Cantidad de equipos = 1

Coeficiente normalizado = 0.8571/1 = 0.8571

Como se observa el valor del coeficiente calculado a mano es 0.8571, y coincide con el calculado por la herramienta, el cual se señala en la figura anterior.

Es por ello que para formar equipos heterogéneos hay que maximizar el valor de la función objetivo, ya que mientras más heterogéneo es el equipo más cercano a 1 es el valor del coeficiente

Caso de prueba 2: Formación de equipo homogéneo con factor Religión

Objetivo: Comprobar que se calcula correctamente el coeficiente de la función objetivo relacionada con el factor Religión y que se forma un equipo homogéneo.

Como se muestra en la Figura 16 la herramienta formó un equipo homogéneo, ya que está compuesto por diez personas de diferentes religiones:

Jesús Manuel Castellanos Reynaldo (Judaísmo), Hans Concepción Álvarez (Judaísmo), Raúl Alejandro De Jesús León (Agnóstico), Claudia Gonzales Fernández (Judaísmo), Kendry Owen Estévez Rodríguez (Yoruba), Eduardo Enrique Fonseca Heredia (Yoruba), Jacqueline Menéndez de San Pedro Guillen (Ateo), Adrián Báez Domínguez (Bautista), Alain Hernández Benavides (Agnóstico), Laura Mary Rosales Domínguez (Yoruba).

Propuesta de equipos

Obtener propuesta

Salvar equipo

Fijar persona

Propuesta: Equipo Homogéneo Religión: 0.4286

Grupo 13

Jefe de Brigada Propuesto

Jesús Manuel Castellanos Reynaldo

Miembro

Hans Concepcion Alvarez

Raul Alejandro De Jesús león

Claudia Gonzalez Fernandez

Kendry Owen Estévez Rodríguez

Eduardo Enrique Fonseca Heredia

Jacqueline Menéndez de San Pedro Guillén

Adrian Baez Dominguez

Alain Hernández Benavides

Laura Mary Rosales Domínguez

Figura 16: Propuesta de equipo homogéneo (Factor Religión).

Debido que existe diversidad en cuanto al factor demográfico de la religión, la cantidad de religiones diferentes en el equipo es 5.

El valor mínimo de religiones diferentes es 1.

El valor máximo es 8 ya que la cantidad de personas del equipo es mayor que la cantidad de religiones diferentes que existe en el grupo (10 personas en el equipo > 8 religiones).

valor = (cant de religiones diferentes de cada equipo – valor mín) / (valor máx – valor mín)

valor = (5 - 1) / (8 - 1) 4 / 7 = 0.5714

Cantidad de equipos =1

Coeficiente normalizado = 0.5714 / 1 = 0.5714

Como la herramienta está configurada para maximizar las funciones objetivo, y para formar equipos homogéneos hay que minimizar el valor del coeficiente normalizado, es preciso restar (1 - coeficiente normalizado). En este caso el resultado es 1 – 0.5714 = 0.4286. Por lo tanto, el valor del coeficiente calculado a mano coincide con el calculado por la herramienta, como se resalta en la figura anterior.

3.2.2 Comprobar el funcionamiento de las funciones objetivo relacionadas con el factor Religión (Balance)

Para calcular el coeficiente normalizado para las funciones objetivo relacionados con el factor Religión se emplea la siguiente fórmula:

Coeficiente normalizado = valor / (cantidad de equipos * por la cantidad de religiones diferentes en los estudiantes de primer año)

Donde:

Valor = $\sum_{i=1}^n |\bar{X} - \text{cantidad de religiones diferentes de cada equipo}|$

$\bar{X} = \sum_{i=1}^n |\text{cantidad de religiones diferentes de cada equipo}| / \text{cantidad de equipos}$

Caso de prueba 3: Balancear equipos heterogéneos con factor Religión

Objetivo: Comprobar que se calcula correctamente el coeficiente de la función objetivo relacionada con el factor Religión y que se forman equipos de forma heterogénea.

En el Grupo 13 está compuesto por:

El grupo 13 está compuesto por: Alfredo Hernández Rodríguez (Yoruba), José Janquiel López Rondón (Agnóstico), Alejandra Sánchez Cabrera (Ateo), Nicole Álvarez Matos (Católico), Alejandro Estrada Suarez (Pentecostal), Osvaldo David García Reyes (Pentecostal), Katherine Ramírez Hidalgo (Agnóstico), Dairo Dasiel Vilella Durán (Testigo de Jehová), Oscar Dixon Hernández (Judaísmo), Adrián Suarez Padrón (Yoruba).

El Grupo 14 quedó compuesto por:

El grupo 14 está compuesto por: Andy Carlos Nápoles Pérez (Agnóstico), Ailen Delgado Castro Quirós (Católico), Daríel Mosquera Molina (Yoruba), Melany Rodríguez Arias (Ateo), Amanda Acevedo Madrazo (Católico), Kristian Carreño Contino (Bautista), Ernesto Alejandro Carralero Conde (Bautista), Alejandro García Rodríguez (Pentecostal), Rene Duran Losada (Judaísmo), Elena Rita Gonzales Llorente (Yoruba).

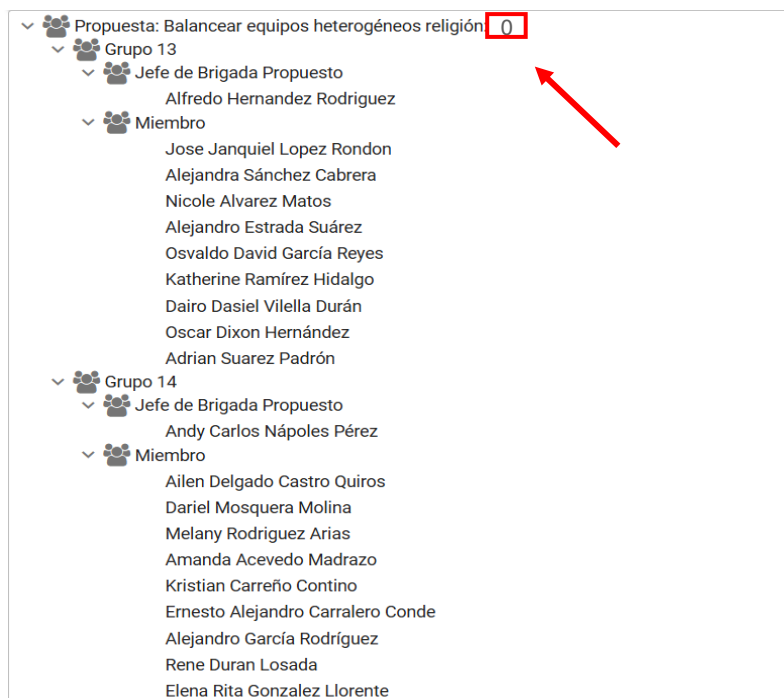


Figura 17: Propuesta de equipos balanceados tomando en cuenta el factor Religión (Heterogéneo).

Cantidad de equipos: 2

Tanto en el grupo 13 como en el 14 existe 7 religiones diferentes:

$$\text{Media} = (7 + 7) / 2 = 7$$

$$\text{Coeficiente normalizado} = ([7 - 7] + [7 - 7]) / 2 * 8 = 0 / 16 = 0$$

Como se observa el valor del coeficiente calculado a mano es 0 y coincide con el calculado por la herramienta, que se encuentra señalado en la figura anterior.

Caso de prueba 4: Balancear equipos homogéneos con factor Religión

Objetivo: Comprobar que se calcula correctamente el coeficiente de la función objetivo relacionada con el factor Religión y que se balance los equipos de forma homogénea.

Como se muestra en la Figura 18 la herramienta formó dos equipos homogéneos:

El grupo 13 está compuesto por:

Adrián Suarez Padrón (Yoruba), Denilson Carmona Machado (Bautista), Gisselle Blanco Almeida (Bautista), Dariel Mosquera Molina (Yoruba), Dariel Pupo Carmenate (Ateo), Luis Emilio Llorens Couso (Pentecostal), Felix Manuel Velázquez Sánchez (Testigo de Jehová), Ever Azcuy Saborit (Bautista), Darío Gonzales Morando (Agnóstico), Sandro Torriente Castaño (Testigo de Jehová).

y el grupo 14 está compuesto:

Daniel Alejandro Corría Montanés (Católico), Kendry Owen Estévez Rodríguez (Yoruba), Eduardo Alejandro Gonzales Martell (Pentecostal), Daniel José Bátiz Gonzales (Judaísmo), Denilson Carmona Machado (Bautista), Alberto Rodríguez Castro (Testigo de Jehová), Adrián Suarez Padrón (Yoruba), Daniel Jesús Castro de la Nuez (Católico), Julio Sergio López Quirós (Yoruba).

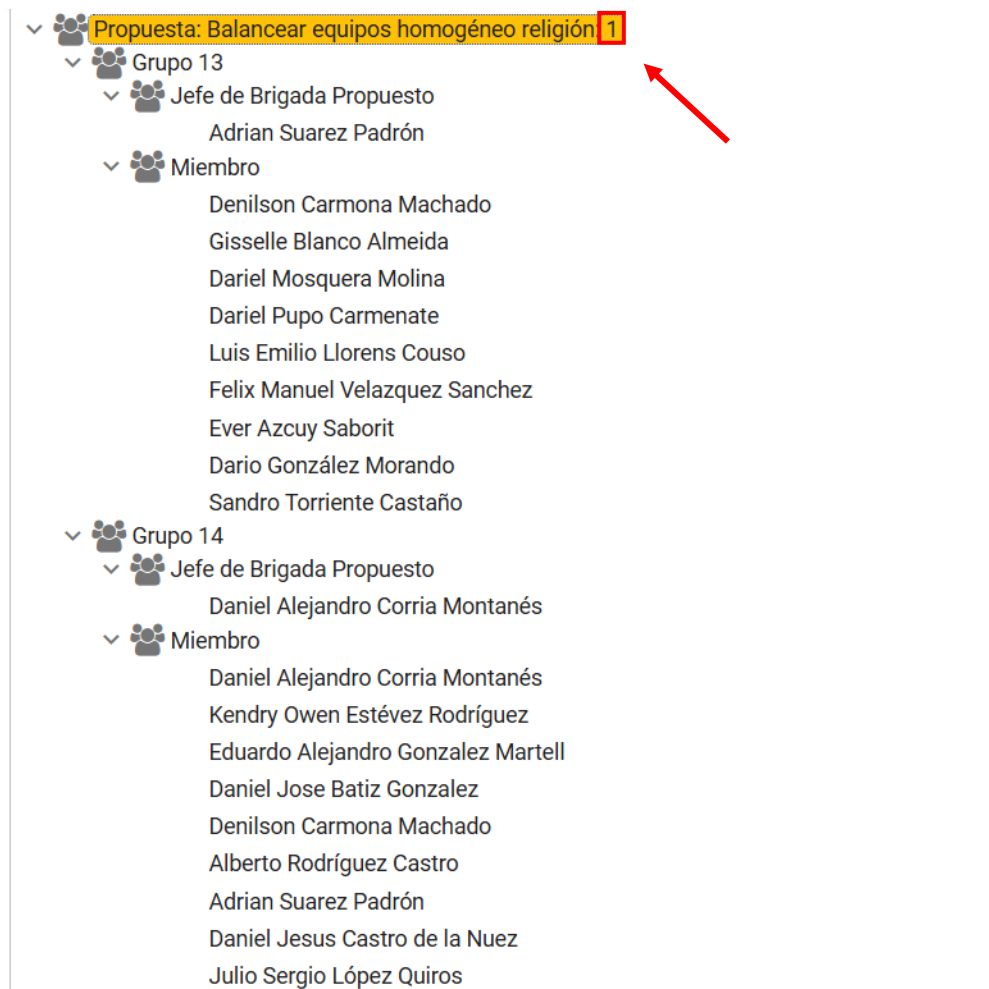


Figura 18: Propuesta de equipos balanceados tomando en cuenta el factor Religión (Homogéneo)

Cantidad de equipos: 2

Tanto en el grupo 13 como en el 14 existen 6 religiones distintas:

$$\text{Media} = (6 + 6) / 2 = 6$$

$$\text{Coeficiente normalizado} = ([6 - 6] + [6 - 6]) / 2 * 8 = 0 / 16 = 0$$

Como la herramienta está configurada para maximizar las funciones objetivo, y para formar equipos homogéneos hay que minimizar el valor del coeficiente normalizado, es preciso restar (1 - coeficiente normalizado). En este caso el resultado es 1 - 0 = 1. Por lo tanto, el valor del coeficiente calculado a mano coincide con el calculado por la herramienta, como se resalta la figura anterior.

3.2.3 Comprobar el funcionamiento de las funciones objetivo relacionadas con el factor Grupo de Edades

Para calcular el coeficiente normalizado para las funciones objetivo relacionadas con el factor Religión se emplea la siguiente fórmula:

Coeficiente normalizado = valor / cantidad de equipos

valor = (cantidad de religiones diferentes en el equipo – valor min) / (valor máx – valor min)

El valor mínimo es 1 ya que en el peor de los casos al menos hay una religión en el equipo.

El valor máximo es la cantidad de religiones diferentes en la organización si la cantidad de personas del equipo es mayor o igual a la cantidad de nacionalidades diferentes en la organización, sino el valor máximo es la cantidad de personas del equipo.

Caso de prueba 5: Formación de equipo heterogéneo con factor Grupo de Edad

Objetivo: Comprobar que se calcula correctamente el coeficiente de la función objetivo relacionada con el factor grupo de edad y que se forma un equipo heterogéneo.

Como se muestra en la Figura 19 la herramienta formó un equipo heterogéneo, ya que está compuesto por diez personas de diferentes grupos de edades, la cantidad de grupos de edades en el equipo 4:

José Janquiel López Rondón (Plan C, 19 años), Carlos Alberto Puyalena Placeres (Plan C 19 años), Waldo Fernández Farinas (Plan D, 18 años), Roció Fernández Carrero (Plan D, 18 años), Darío Gonzales Morando (Plan B, 20 años), Rafael Castellanos Velázquez (Plan D, 18 años), Mónica Rodríguez Delgado (Plan A, 21 años), Adrián Báez Domínguez (Plan D, 18 años), Dariel Pupo Carmenate (Plan D, 18 años), Frank Mesa Domínguez (Plan B, 20 años).

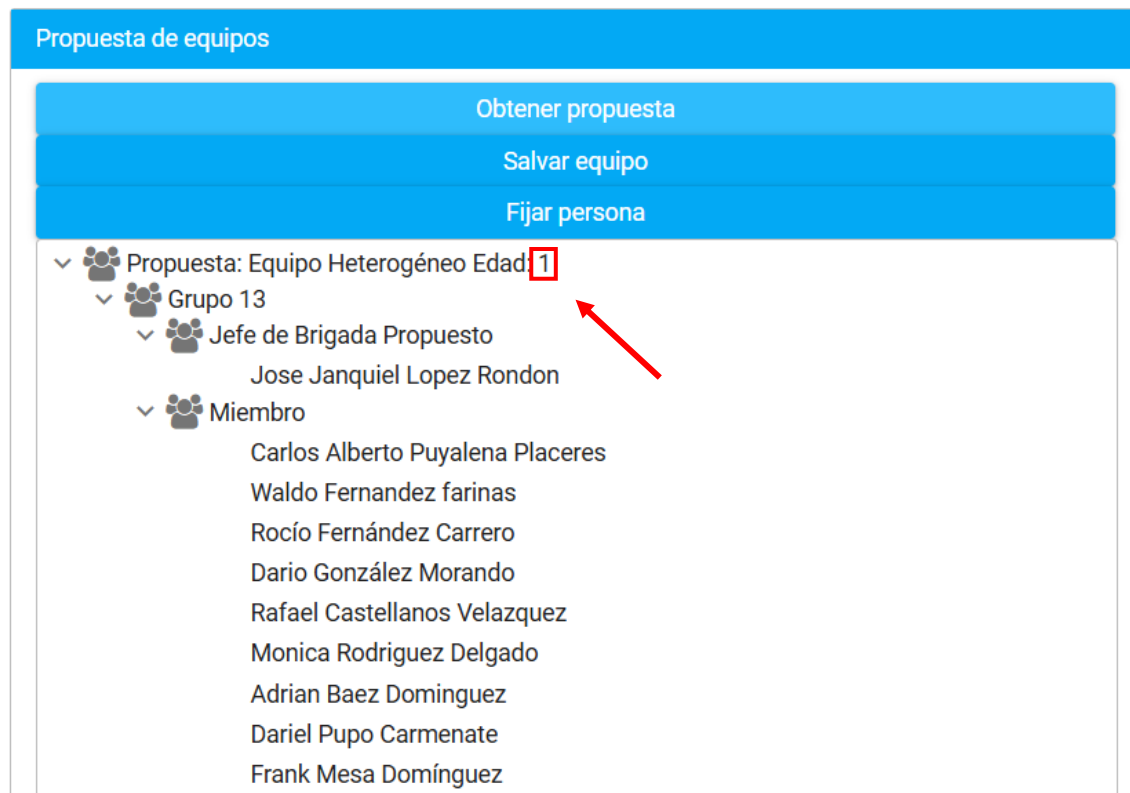


Figura 19: Propuesta de equipo Heterogéneo para el factor Grupo de Edades

Debido a que existe diversidad en cuanto al factor grupo de edad, la cantidad de grupo de edades diferentes en el equipo es 4.

El valor mínimo de grupo de edades es 1.

El valor máximo es 4 ya que la cantidad de personas del equipo es mayor a la cantidad de grupos de edades diferentes que existen en el grupo (4 grupos – 10 personas en el equipo).

$$\text{valor} = (\text{cant de grupo de edades diferentes de cada equipo} - \text{valor mínimo}) / (\text{valor máximo} - \text{valor mínimo})$$

$$\text{valor} = (4 - 1) / (4 - 1) = 3 / 3 = 1$$

Cantidad de equipos = 1

Coeficiente normalizado = $1 / 1 = 1$

Como se observa el valor del coeficiente calculado a mano es 1, y coincide con el calculado por la herramienta, el cual se señala en la figura anterior. Es por ello que para formar equipos heterogéneos hay que maximizar el valor de la función objetivo, ya que mientras más heterogéneo es el equipo más cercano a 1 es el valor del coeficiente.

Caso de prueba 6: Formación de equipo homogéneo con Factor Grupo de Edades

Objetivo: Comprobar que se calcula correctamente el coeficiente de la función objetivo relacionada con el factor grupo de edad y que se forma un equipo homogéneo.

Como se muestra en la Figura 20 la herramienta formó un equipo homogéneo compuesto por diez personas:

Gisselle Blanco Almeida (Plan D, 18 años), Liset Arias Fonseca (Plan D, 18 años), Diego Basilio Zada Fernández (Plan A, 21 años), Fabian Alejandro Reyes Montero (Plan D, 18 años), Mónica Rodríguez Delgado (Plan A), Alessandro Campos Diaz (Plan D, 18 años), Waldo Fernández Farinas (Plan D, 18 años), Brenda Silvia Gutiérrez Herrero (Plan D, 18 años), Alejandro Aguirre García (Plan D, 18 años), Diego Rodríguez Arce (Plan B, 20 años).



Figura 20: Propuesta de equipo Homogéneo para el factor Grupo de Edad.

Debido a que existe diversidad en cuanto al factor grupo de edad, la cantidad de grupo de edades diferentes en el equipo es 3.

El valor mínimo de grupo de edades es 1.

El valor máximo es 4 ya que la cantidad de personas del equipo es mayor a la cantidad de grupos de edades diferentes que existen en el grupo (4 grupos – 10 personas en el equipo).

valor = (cant de grupo de edades diferentes de cada equipo – valor mínimo) / (valor máximo – valor mínimo)

valor = (3 - 1) / (4 - 1) = 2 / 3 = 0.6667

Cantidad de equipos = 1

Coeficiente normalizado = 0.667 / 1 = 0.6667

Como la herramienta está configurada para maximizar las funciones objetivo, y para formar equipos homogéneos hay que minimizar el valor del coeficiente normalizado, es preciso restar (1-coeficiente normalizado). En este caso el resultado es $1 - 0.6667 = 0.333$. Por lo tanto, el valor del coeficiente calculado a mano coincide con el calculado por la herramienta, como se resalta la figura anterior.

3.2.3 Comprobar el funcionamiento de las funciones objetivo relacionadas con el factor Grupo de Edades (Balance)

Para calcular el coeficiente normalizado para las funciones objetivo relacionados con el factor Religión se emplea la siguiente fórmula:

Coeficiente normalizado = valor / (cantidad de equipos * por la cantidad de religiones diferentes en los estudiantes de primer año)

Donde:

Valor = $\sum_{i=1}^n |\bar{X} - \text{cantidad de religiones diferentes de cada equipo}|$

$\bar{X} = \sum_{i=1}^n |\text{cantidad de religiones diferentes de cada equipo}| / \text{cantidad de equipos}$

Caso de prueba 7: Balancear equipos heterogéneos con factor Grupo de Edad

Objetivo: Comprobar que se calcula correctamente el coeficiente de la función objetivo relacionada con el factor grupo de edad y que balancea los equipos de forma heterogénea.

Como se muestra en la Figura 21 la herramienta formó dos equipos heterogéneos:

El grupo 13 está compuesto por: Nicole Alvares Matos (Plan D, 18 años), Brian Serrano Alfonso (Plan C, 19 años), Daniel Zuazo Tejero (Plan C, 19 años), Daniel Alejandro Corría Montanés (Plan D, 18 años), Kevin Gallardo González (Plan D, 18 años), Miguel Ángel López García (Plan C, 19 años), Ernesto Matamoros Fernández (Plan B, 20 años), Luis Javier Pérez Ruiz (Plan D, 18 años), Ismaray Del Sol Cruz (Plan B, 20 años), Daniel Alejandro Deyne Rodríguez (Plan A, 21 años).

El grupo 14 está compuesto por: Juliette García Campos (Plan D, 18 años), Jacqueline Menéndez de San Pedro Guillen (Plan C, 19 años), Anyeleni Lara Santana (Plan B), Kevin Gallardo González (Plan D, 18 años), Alain Hernández Benavides (Plan C, 19 años), Félix Manuel Velázquez Sánchez (Plan B, 20 años), Alfredo Hernández Rodríguez (Plan C, 19 años), Lissandra María Martínez Llorente (Plan A, 21 años), Gisselle Blanco Almeida (Plan D, 18 años), Dashiel Beltrán Varela (Plan D, 18 años).

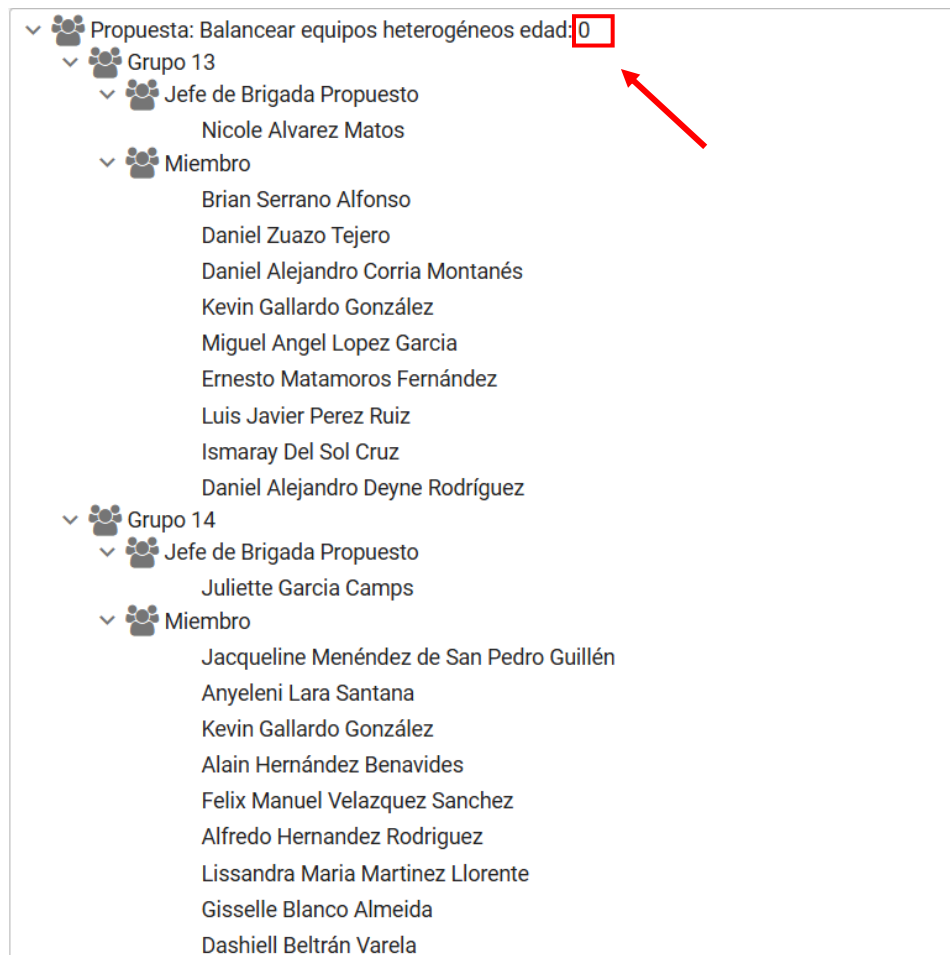


Figura 21: Propuestas de equipos balanceados tomando en cuenta el factor Grupo de Edad (Heterogéneo).

Cantidad de equipos: 2

Tanto en el grupo 13 como en el grupo 14 existe 4 grupos de edades distintas.

$$\text{Media} = (4 + 4) / 2 = 4$$

$$\text{Coeficiente normalizado} = ([4 - 4] + [4 - 4]) / 2 * 4 = 0 / 8 = 0$$

Como se observa el valor del coeficiente calculado a mano es 0 y coincide con el calculado por la herramienta, que se encuentra señalado en la figura anterior.

Caso de prueba 8: Balancear equipos homogéneos con factor Grupo de Edad

Objetivo: Comprobar que se calcula correctamente el coeficiente de la función objetivo relacionada con el factor grupo de edad y que balancea los equipos de forma homogéneos.

Como se muestra en la Figura 22 la herramienta formó dos equipos homogéneos:

El grupo 13 está compuesto por: Julio Cesar Polo Mesa (Plan A, 21 años), Alfredo Hernández Rodríguez (Plan C, 19 años), Antonio Lamorú Varela (Plan B, 20 años), Raúl Alejandro De Jesús León (Plan B, 20 años), Alejandro Montes Hernández (Plan C, 19 años), Katherine Ramírez Hidalgo (Plan B, 20 años), Daniel José Bátiz Gonzales (Plan D, 18 años), Daniel Alejandro Corría Montanés (Plan D, 18 años), Roció Fernández Carrero (Plan D, 18 años), Eduardo Enrique Fonseca Heredia (Plan A, 21 años).

El grupo 14 está compuesto por: Eduardo Enrique Fonseca Heredia (Plan A, 21 años), Dariel Mosquera Molina (Plan B, 20 años), Ailen Delgado Castro Quirós (Plan D, 18 años), Diego Alfonso Criado (Plan D, 18 años), Rafael Castellanos Velázquez (Plan D, 18 años), Patricia Barrios Pupo (Plan B, 20 años), Melany Rodríguez Arias (Plan D, 18 años), Alejandro García Rodríguez (Plan D, 18 años), José Janquiel López Rondón (Plan C, 19 años), Daniel Zuazo Tejero (Plan C, 19 años).

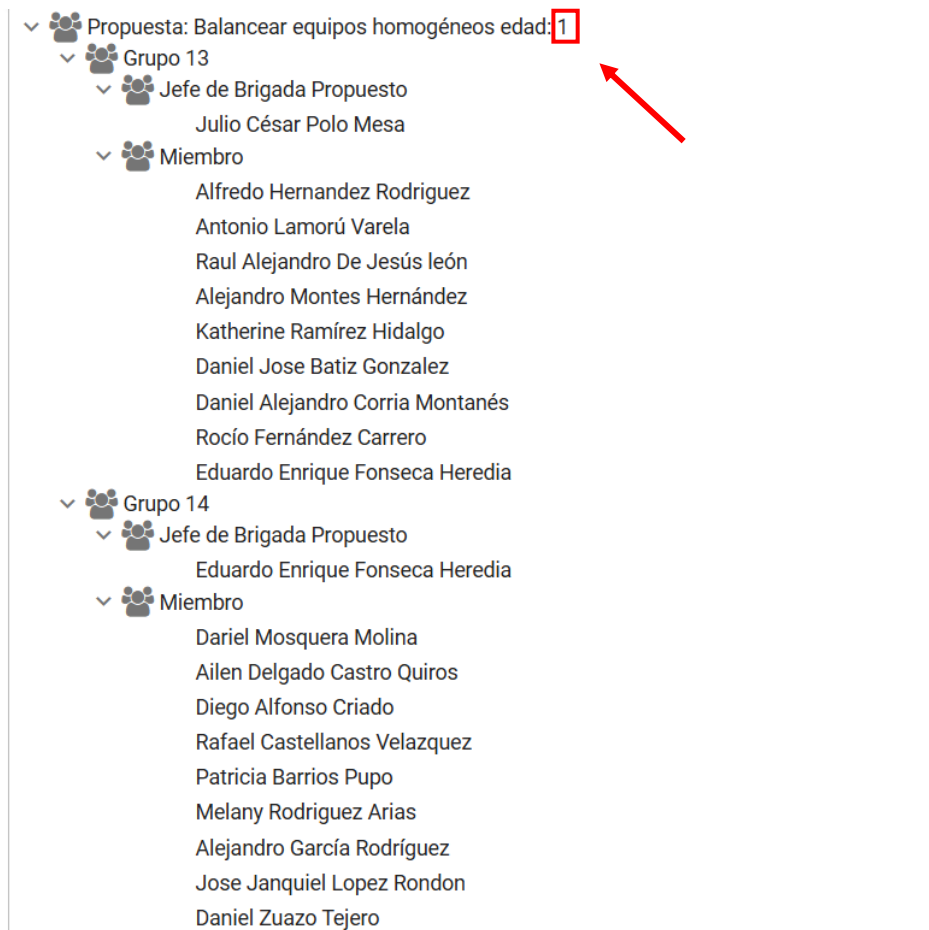


Figura 22: Propuesta de equipos balanceados tomando en cuenta el factor Grupo de Edad (Homogéneo).

Cantidad de equipos: 2

Tanto en el grupo 13 como en el grupo 14 existe 4 grupos de edades distintas.

$$\text{Media} = (4 + 4) / 2 = 4$$

$$\text{Coeficiente normalizado} = ([4 - 4] + [4 - 4]) / 2 * 4 = 0 / 8 = 0$$

Como la herramienta está configurada para maximizar las funciones objetivo, y para formar equipos homogéneos hay que minimizar el valor del coeficiente normalizado, es preciso restar (1-coeficiente normalizado). En este caso el resultado es $1 - 0 = 1$. Por lo tanto, el valor del coeficiente calculado a mano coincide con el calculado por la herramienta, como se resalta la figura anterior.

3.3 Refactorización y mejoras en la Calidad estática del código

SonarQube es una herramienta de código abierto ampliamente utilizada para la inspección continua de la calidad del código. Su principal función es analizar el código fuente en busca de posibles errores, vulnerabilidades, complejidad ciclomática y, en particular, código duplicado. Entre sus ventajas destacan su capacidad para integrarse con múltiples lenguajes de programación, su interfaz intuitiva que facilita la identificación de problemas, y su enfoque en la mejora continua mediante métricas claras y detalladas. Sin embargo, también presenta algunas desventajas, como la necesidad de una configuración inicial compleja y la posibilidad de generar falsos positivos que requieren revisión manual [92].

En este estudio, se decidió evaluar la calidad del código y, específicamente, el nivel y la cantidad de código duplicado presente en el proyecto. Esta decisión se tomó debido a que, durante el desarrollo de sus versiones, han participado diversos estudiantes, lo que ha generado inconsistencias en la calidad del código. La falta de un estándar unificado y la rotación de desarrolladores han contribuido a la aparición de duplicaciones, lo que afecta negativamente la mantenibilidad y escalabilidad del proyecto.

De igual forma se realizó una evaluación posterior al desarrollo de la nueva API Rest de TeamSoft+, para que, mediante SonarQube se demuestre cómo influyó la refactorización del proyecto antiguo y la creación del nuevo en la calidad estática del código.

3.3.1 Análisis con SonarQube

Previo al proceso de migración y refactorización, se realizó un análisis estático del código de la versión original de TeamSoft+. Este análisis permitió identificar problemáticas críticas en la base de código, entre las cuales destacó un alto nivel de duplicación. Como se muestra en la Figura 10, de un total de 44 000 líneas de código analizadas, 6 424 correspondían a código duplicado, lo que representaba un 14,6 % del total. Este resultado evidenció la necesidad urgente de aplicar técnicas de refactorización para mejorar la mantenibilidad y reducir la deuda técnica acumulada.

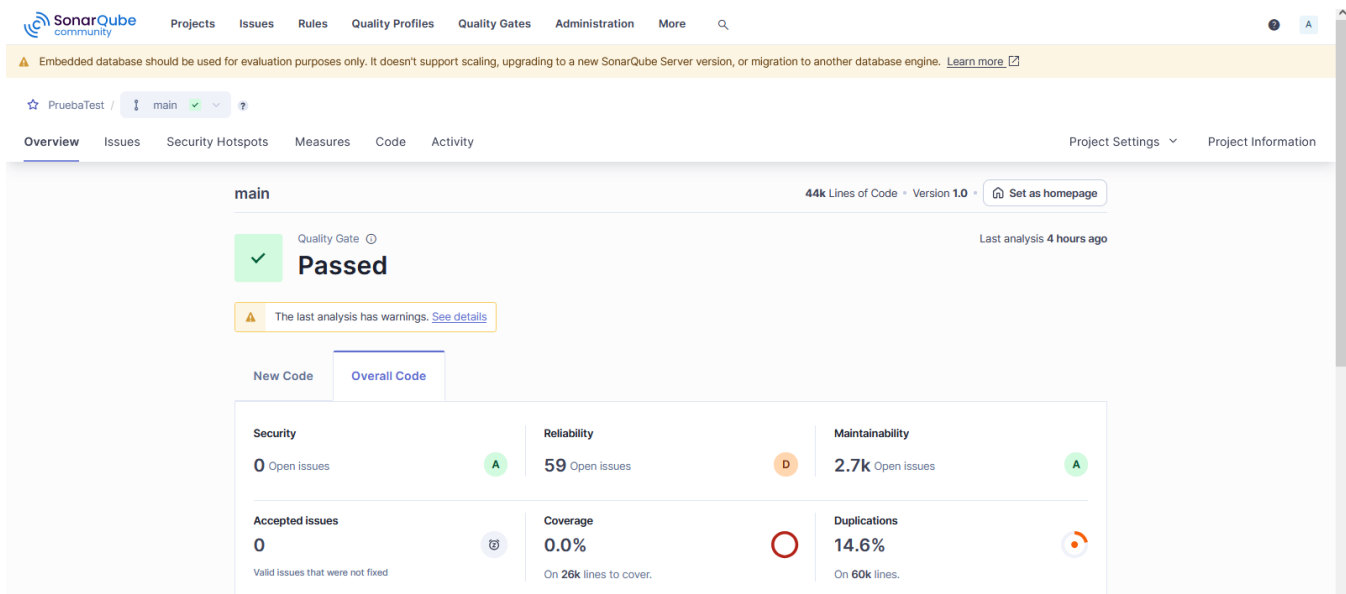


Figura 23: Vista general de las métricas arrojadas por SonarQube de la versión anterior de TeamSoft+

3.3.2 Técnicas de refactorización ejecutadas

Durante el proceso de migración se aplicaron sistemáticamente técnicas de refactorización con el objetivo de mejorar la estructura del código sin alterar su comportamiento funcional. Se aprovechó el desarrollo de la nueva aplicación para corregir y eliminar problemas identificados. Entre las técnicas aplicadas se encuentran:

- **Extracción de métodos:** Se identificaron métodos con fragmentos de código comunes y se fraccionaron en métodos independientes y reutilizables.
- **Eliminación de código duplicado:** Se unificaron secciones de código repetitivas en componentes únicos.
- **Limpieza de código innecesario:** Se eliminaron líneas de código comentadas, variables sin uso y métodos obsoletos.
- **Simplificación de estructuras:** Se reorganizaron clases y métodos para reducir su complejidad y mejorar la legibilidad.

3.3.3 Resultados del análisis Post-Refactorización

Una vez finalizada la migración de las funcionalidades principales (gestión de nomencladores, personas, creación de equipos, entre otras) se realizó un nuevo análisis con SonarQube sobre el código del *back-end* de la API REST. Los resultados mostraron una mejora significativa en la calidad estática del código: el total de líneas de código se redujo a 16 000, de las cuales solo 1776 correspondían a código duplicado, lo que representa un 11,1 % del total. Esta reducción del 3,5 % en la duplicación demuestra un avance tangible en la limpieza y eficiencia del código, confirmando que la refactorización progresiva durante la migración fue efectiva para atacar la deuda técnica y sentar las bases de un sistema más escalable y mantenible.

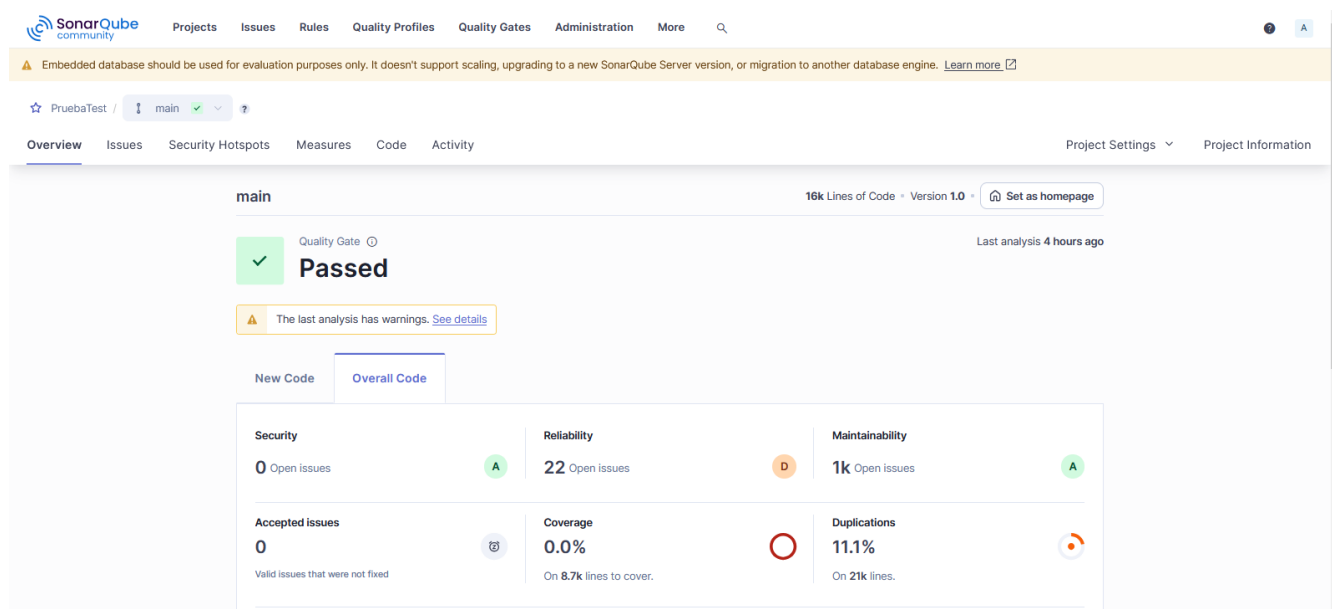


Figura 24: Vista general de las métricas arrojadas por SonarQube en el código bak-end de la nueva versión de TeamSoft+.

3.4 Diseño y ejecución de pruebas para validar el correcto funcionamiento de los *endpoints* desarrollados

Para validar el correcto funcionamiento de los controladores y servicios desarrollados se elaboraron casos de prueba que validaran su funcionamiento.

En las Tablas 1, 2 y 3 se presentan casos de pruebas para el factor de grupo de edad.

Caso de Uso			Gestionar Grupo de Edad (crear)	
Descripción breve del caso de uso			El caso de uso se inicia por el gestor de recursos humanos, cuando se registran los datos de cada Grupo de edad (nombre de la religión, edad mínima, edad máxima), permitiendo su registro, modificación y eliminación	
Desarrollador / Probador			Dennis Serafín Rodríguez Armenteros / Leyder Ignacio Herrera Bello	
Fecha de diseño / Fecha de ejecución			3 de noviembre de 2025 / 5 de noviembre de 2025	
Objetivo de la prueba			Verificar que la gestión funcione correctamente	
Combinaciones de valores de entrada			Resultados	
CP	Escenario	Variable / Valor	Esperados	Reales
1	Registro Exitoso	{ "ageGroupName": "Plan A", "maxAge": 21, "minAge": 21 }	{ "id": 217, "ageGroupName": "Plan A", "maxAge": 21, "minAge": 21 } con código 201 Created	{ "id": 217, "ageGroupName": "Plan A", "maxAge": 21, "minAge": 21 } con código 201 Created
2	Campos vacíos	{ "ageGroupName": "", "maxAge": 21, "minAge": 21 }	{ "ageGroupName": "Age group name is required" } con código 400 Bad Request	{ "ageGroupName": "Age group name is required" } con código 400 Bad Request
3	El grupo de edad ya existe	{ "ageGroupName": "Plan B", "maxAge": 22, "minAge": 20 }	{ "Error: ": "Error saving age group: The age range (20-22) overlaps with an existing age group" } con código 400 Bad Request	{ "Error: ": "Error saving age group: The age range (20-22) overlaps with an existing age group" } con código 400 Bad Request
4	Inserta valores no permitidos	{ "ageGroupName": "Plan B", "maxAge": 22, "minAge": -20 }	{ "minAge": "Minimum age must be at least 0" } con código 400 Bad Request	{ "minAge": "Minimum age must be at least 0" } con código 400 Bad Request

Tabla 1: Caso de prueba del Caso de Uso Gestionar Grupo de Edad (Crear).

Caso de Uso			Gestionar Grupo de Edades (editar)	
Descripción breve del caso de uso			El caso de uso se inicia por el gestor de recursos humanos, cuando se registran los datos de cada Grupo de edad (nombre de la religión, edad mínima, edad máxima), permitiendo su registro, modificación y eliminación	
Desarrollador / Probador			Dennis Serafín Rodríguez Armenteros / Leyder Ignacio Herrera Bello	
Fecha de diseño / Fecha de ejecución			3 de noviembre de 2025 / 5 de noviembre de 2025	
Objetivo de la prueba			Verificar que la gestión funcione correctamente	
Combinaciones de valores de entrada			Resultados	
CP	Escenario	Variable / Valor	Esperados	Reales
1	Registro Exitoso	{ "ageGroupName": "Plan A", "maxAge": 21, "minAge": 20 } para id = 217	{ "id": 217, "ageGroupName": "Plan A", "maxAge": 21, "minAge": 21 } con código 200 OK	{ "id": 217, "ageGroupName": "Plan A", "maxAge": 21, "minAge": 20 } con código 200 OK
2	Campos vacíos	{ "ageGroupName": "", "maxAge": 21, "minAge": 20 } para id = 217	{ "ageGroupName": "Age group name is required" } con código 400 Bad Request	{ "ageGroupName": "Age group name is required" } con código 400 Bad Request
3	El grupo de edad ya existe	{ "ageGroupName": "Plan B", "maxAge": 21, "minAge": 20 } para id = 218	{ "Error: ": "Error saving age group: The age range (20-21) overlaps with an existing age group" } con código 400 Bad Request	{ "Error: ": "Error saving age group: The age range (20-21) overlaps with an existing age group" } con código 400 Bad Request
4	Inserta valores no permitidos	{ "ageGroupName": "Plan A", "maxAge": -21, "minAge": 20 } para id = 217	{ "maxAge": "Maximum age must be at least 0", "ageRangeValid": "Minimum age must be less than or equal to maximum age" } con código 400 Bad Request	{ "maxAge": "Maximum age must be at least 0", "ageRangeValid": "Minimum age must be less than or equal to maximum age" } con código 400 Bad Request

Tabla 2: Caso de prueba del Caso de Uso Gestionar Grupo de Edad (Editar).

Caso de Uso		Gestionar Grupo de Edades (eliminar)		
Descripción breve del caso de uso		El caso de uso se inicia por el gestor de recursos humanos, cuando se registran los datos de cada Grupo de edad (nombre de la religión, edad mínima, edad máxima), permitiendo su registro, modificación y eliminación		
Desarrollador / Probador		Dennis Serafín Rodríguez Armenteros / Leyder Ignacio Herrera Bello		
Fecha de diseño / Fecha de ejecución		3 de noviembre de 2025 / 5 de noviembre de 2025		
Objetivo de la prueba		Verificar que la gestión funcione correctamente		
Combinaciones de valores de entrada			Resultados	
CP	Escenario	Variable / Valor	Esperados	Reales
1	Eliminación Exitosa	id = 218	Age group deleted con código 200 OK	Age group deleted con código 200 OK
2	No eliminación del grupo de edad por que no existe	id = 219	{ "Error: ": "Age group not found with ID: 219" } con código 404 Not Found	{ "Error: ": "Age group not found with ID: 219" } con código 404 Not Found

Tabla 3: Caso de prueba del Caso de Uso Gestionar Grupo de Edad (Eliminar).

3.5 Pruebas de rendimiento

Para validar las mejoras en el rendimiento derivadas de la migración hacia una arquitectura basada en API REST, se diseñó y ejecutó un conjunto de pruebas comparativas entre la versión anterior de TeamSoft+ (arquitectura monolítica) y la nueva versión (con backend en Spring Boot). Posteriormente, se realizaron peticiones HTTP equivalentes a ambos sistemas utilizando el software Postman, el cual permite medir el tiempo que tarda cada API en procesar una solicitud y devolver una respuesta. Estas pruebas se ejecutaron en un entorno controlado con un procesador Intel i3-13100 y 8 GB de RAM.

En la Tabla 4 se muestran los tiempos de respuesta de 10 peticiones consecutivas, así como el tiempo promedio, evidenciando una mejora significativa en la eficiencia de la nueva versión.

Funcionalidad	Tiempo de Respuesta (ms)	
	TeamSoft +	API
Crear grupo de persona	175	52
Editar grupo de persona	222	73
Crear religión	1340	28
Editar religión	949	52
Crear rol	297	111
Editar rol	504	76
Crear nivel	167	146
Editar nivel	95	78
Crear raza	606	150
Editar raza	667	146
Promedio	502,2	91,2

Tabla 4: Resultados de pruebas de rendimiento realizadas a ambas versiones de la aplicación.

3.6 Conclusiones parciales

Una vez culminado el capítulo se arriban a las siguientes conclusiones:

1. Los resultados de las pruebas ejecutadas permitieron comprobar que los coeficientes de las funciones objetivo implementadas se calculan correctamente.
2. Los datos arrojados por SonarQube fueron claves para demostrar la efectividad de la refactorización desarrolla, demostrando una mejoría significativa en los tiempos de respuestas de ambas versiones.
3. Los casos de pruebas diseñados demostraron el correcto funcionamiento de las funcionalidades implementadas en la API

Conclusiones

1. La formación de equipos es un proceso crítico para el éxito de las organizaciones, que debe considerar múltiples factores tanto individuales como colectivos. La incorporación de factores demográficos como la religión y el grupo de edades enriquece sustancialmente este proceso, permitiendo una gestión más equilibrada y consciente de la diversidad.
2. La incorporación de los factores demográficos “religión” y “grupo de edades” convierte a TeamSoft+ en una herramienta más robusta y completa, permitiendo gestionar proactivamente posibles líneas de falla y fomentando la cohesión y productividad en los equipos.
3. El rediseño de la base de datos y la migración hacia una arquitectura basada en API REST sentaron las bases técnicas necesarias para la transición de TeamSoft+ hacia un modelo de negocio híbrido (SaaS/PaaS), facilitando la escalabilidad y el despliegue multi-cliente.
4. La nueva arquitectura, con una clara separación entre *front-end* y *back-end*, mejora significativamente la mantenibilidad y escalabilidad del sistema, permitiendo que ambos componentes evolucionen de manera independiente.
5. El uso de SonarQube fue fundamental para identificar y corregir problemas de calidad estática del código, como duplicaciones y código inutilizado, lo que contribuyó a reducir la deuda técnica y mejorar la mantenibilidad del sistema.
6. Las pruebas funcionales y de rendimiento realizadas permitieron validar el correcto funcionamiento de las nuevas funcionalidades y de los *endpoints* desarrollados, confirmando la estabilidad y eficiencia de la solución.
7. La validación mediante casos de prueba exhaustivos y la posterior corrección de errores garantizaron que la herramienta funcione de manera correcta y confiable en distintos escenarios.
8. Como resultado final, se obtuvo una versión de TeamSoft+ más rápida y eficiente.

Recomendaciones

Con vistas a darle continuidad a este trabajo se recomienda:

1. Aplicar la herramienta para formar equipos en otros contextos reales.
2. Realizar experimentos para evaluar el desempeño de diferentes operadores en la formación de equipos en diferentes contextos.
3. Realizar nuevas pruebas para seguir evaluando y mejorando la calidad estática del código desarrollado.
4. Incorporar nuevas funcionalidades que permitan tomar en cuenta otros factores que puedan fracturar la estructura de los equipos a formar, con el fin de minimizar las fallas.

Referencias Bibliográficas

- [1] D. Pentico, "Assignment problems: A golden anniversary survey," *European Journal of Operational Research*, vol. 176, no. 2007, pp. 774–793, 2007.
- [2] A. Zzakarian and A. Kusiak, "Forming teams: An analytical approach," *IIE Trans*, vol. 31, no. 1, pp. 85-97, enero 1999 1999.
- [3] M. André, M. G. Baldoquín, and S. T. Acuña, "Formal model for assigning human resources to teams in software projects," *Information and Software Technology*, vol. 53, no. 3, pp. 259-275, 2011.
- [4] F. Ahmed, L.F. Capretz, and P. Campbell, "Evaluating the Demand for Soft Skills in Software Development," *IEEE Computer Society*, 2012.
- [5] S. Liemhetcharat and M. Veloso, "Modeling and learning synergy for team formation with heterogeneous agents," vol. 2, 2012.
- [6] L.F. Capretz, D. Varona, and A. Raza, "Influence of personality types in software tasks choices," *Computers in Human Behavior*, vol. 52, 2015.
- [7] J. Gutiérrez, C. Astudillo, P. Ballesteros-Pérez, D. Mora-Melià, and Candia-Véjar, "The multiple team formation problem using sociometry," *Computers and Operations Research*, vol. 75, no. 2016, pp. 150-162, 2016.
- [8] B. Boehm, "Capther 4. Human Resource Allocation and Scheduling for Software Project Management," in *Software Project Management in a Changing World*, G. Ruhe and C. Wohlin, Eds.: Springer Heidelberg New York Dordrecht London, 2014.
- [9] L. Rampersaud, "TEAMSOF⁺: Sistema para la gestión del trabajo en equipo en el desarrollo de proyectos de software. Versión 3.0. Módulo de asignación de personal," Trabajo de Diploma para optar por el título de Ingeniería en Informática Tesis de Grado, Facultad de Ingeniería Informática, Instituto Superior Politécnico "José Antonio Echeverría", Ciudad de la Habana, Cuba, 2009.
- [10] L.F.Capretz and F. Ahmed, "Why Do We Need Personality Diversity in Software Engineering?," *ACM SIGSOFT Software Engineering Notes*, vol. 35, no. 2, 2010.
- [11] F. Ahmed, L.F. Capretz, P. Campbell, S. Bouktif, and P. Campbell, "Soft Skills and Software Development: A Reflection from Software Industry " *International Journal of Information Processing and Management (IJIPM)*, vol. 4, no. 3, 2013.
- [12] S. Liemhetcharat and M. Veloso, "Allocating training instances to learning agents for team formation," *Auton Agent Multi-Agent Syst*, pp. 1-36, 2017.
- [13] T. Okimoto, N. Schwind, M. Clement, T. Ribeiro, K. Inoue, and P. Marquis, "How to form a task-oriented robust team," presented at the Proceedings of the 14th International Conference on Autonomous Agents and Multiagent Systems (AAMAS), Istanbul, Turkey, May 4-8, 2015, 2015.
- [14] P. Öztürk, K. Rossland, and O. Gundersen, "A multiagent framework for coordinated parallel problem solving," *Appl Intell*, vol. 33, no. 2, pp. 132-143, 2010.
- [15] K. Urakawa and T. Sugawara, "Reorganization of agent networks with reinforcement learning based on communication delay," presented at the IEEE/WIC/ACM International Conferences on Web Intelligence and Intelligent Agent Technology, 2012.

- [16] A. Anagnostopoulos, L. Becchetti, C. Castillo, A. Gionis, and S. Leonardi, "Online team formation in social networks," presented at the International World Wide Web Conference Committee (IW3C2), Lyon, France, April 16–20, 2012, 2012.
- [17] B. Ashenagar, A. Hamzeh, N. Eghlidi, and A. Afshar, "A fast approach for multi-objective team formation in social networks," presented at the IKT2015 7th International Conference on Information and Knowledge Technology, 2015.
- [18] J. Basiri, F. Taghiyareh, and A. Ghorbani, "Collaborative team formation using brain drain optimization: a practical and effective solution," *World Wide Web. Springer Science+Business Media New York* 2017, 2017.
- [19] S. Datta, A. Majumder, and K. Naidu, "Capacitated team formation problem on social networks," presented at the KDD'12, Beijing, China, August 12–16, 2012, 2012.
- [20] F. Farhadi, E. Hoseini, S. Hashemi, and A. Hamzeh, "TeamFinder: A Co-clustering based framework for finding an effective team of experts in social networks," presented at the 2012 IEEE 12th International Conference on Data Mining Workshops 2012.
- [21] M. Kargar and A. An, "Discovering top-k teams of experts with/without a leader in social networks," presented at the CIKM'11, Glasgow, Scotland, UK., October 24–28, 2011, 2011.
- [22] M. Kargar, M. Zihayat, and A. An, "Finding affordable and collaborative teams from a network of experts," 2013.
- [23] Y. Han, Y. Wan, L. Chen, G. Xu, and J. Wu, "Exploiting geographical location for team formation in social coding sites," in *PAKDD 2017, Part I*, vol. 10234 LNAI: Springer International Publishing AG 2017, 2017, pp. 499-510.
- [24] J. Alberola, E. Lval, V. Sanchez-Anguix, and V. Julián, "A general framework for testing different student team formation strategies," 2016.
- [25] J. Alberola, E. DelVal, V. Sanchez-Anguix, A. Palomares, and M. Dolores, "An artificial intelligence tool for heterogeneous team formation in the classroom," *Knowledge-Based Systems*, vol. 101, 2016.
- [26] R. Bredereck, T. Köhler, A. Nichterlein, R. Niedermeier, and G. .Philip, "Using Patterns to Form Homogeneous Teams," *Algorithmica*, vol. 71, no. 2, p. 517, 2015.
- [27] D. Strnad and N. Guid, "A fuzzy-genetic decision support system for project team formation," *Applied Soft Computing*, vol. 10, no. 4, agosto 2010 2010.
- [28] T. Henry, "Forming productive student groups using a massively parallel brute-force algorithm," presented at the Proceedings of the World Congress on Engineering and Computer Science 2013 Vol 1, San Francisco, USA, 23-25 october 2013, 2013.
- [29] S. Mirza. (2020). *Team-Formation-Tool*. Available: <https://github.com/simran-mirza/team-formation-tool>
- [30] P. J. T. Ashkan Ebadi, Lei Zhang, Parisa Rashidi, "DisTeam: A decision support tool for surgical team selection," *ARTMED* 1501, 5-2-2017 2017.
- [31] (november 15). *TeamGen-360*. Available: <https://chiefplings.com/tools/teamgen-360/>
- [32] P. University, "CATME Team-Maker: A Tool for Criterion-Based Team Assignment," Purdue University, 2013.
- [33] T. Machine. (2025, november 15). *Team Machine - SaaS*. Available: <https://marketplace.microsoft.com/en-us/product/web-apps/teammachine.teammachine>
- [34] D. L. Villaón, " Incorporación en Teamsoft+ de funciones objetivo,

restricciones y funcionalidades para facilitar la experimentación y formación de equipos en diferentes contextos," 2021.

- [35] J. A. P. Socorro, "Aplicación del modelo de formación de equipos y la herramienta TEAMSOFT+ en diferentes contextos," 2021.
- [36] Beatriz Renovales Pérez, Derly Otón González Pérez, "Incorporación en Teamsoft+ de funciones objetivo, restricciones y funcionalidades para facilitar la experimentación y formación de equipos en diferentes contextos," 2022.
- [37] Y. C. B. Roidel Montenegro Leyva, "Incorporación a TeamSoft+ de funcionalidades y funciones objetivo asociadas a factores demográficos," Universidad Tecnológica de La Habana "José Antonio Echeverría", 2023.
- [38] D. A. Hicks, *Religion and the Workplace: Pluralism, Spirituality, Leadership*. United States of America: Cambridge University Press, 2003.
- [39] A. Maghen *et al.*, "Spirituality and religiosity of non-directed (altruistic) living kidney donors," *Journal of Clinical Nursing*, Article vol. 27, no. 7-8, pp. 1662-1672, 2018.
- [40] D. C. L. J. K. Murnighan, *DEMOGRAPHIC DIVERSITY AND FAULTLINES: THE COMPOSITIONAL DYNAMICS OF ORGANIZATIONAL GROUPS*. Academy of Management Review, 1998.
- [41] D. van Knippenberg, J. F. Dawson, M. A. West, and A. C. Homan, "Diversity faultlines, shared objectives, and top management team performance," *Human Relations*, Article vol. 64, no. 3, pp. 307-336, 2011.
- [42] J. Z. Q. L. Y. Zhang, "Team faultline: types, configuration and influence – evidence from China," vol. 28, no. 3, Accessed on: november, 2025
- [43] V. Yannibelli and A. Amandi, "Forming well-balanced collaborative learning teams according to the roles of their members: An evolutionary approach," presented at the CINTI 2011 - 12th IEEE International Symposium on Computational Intelligence and Informatics, Budapest, Hungary, 21-22 november, 2011, 2011.
- [44] R. D. J. Fajardo. (2018, 19-1-18). *BiCIAM Library*. Available: <http://modo.ugr.es/algorithmportfolio/#biciam>
- [45] B. T. Yang Han, Xian-Miao Li, Guo-Song-Rui Yang, Li Yang, "Research on the Relationship Between Informational Team Faultline and Team Resilience- Team Leader Member Exchange and Team Member Exchange as Mechanism," *Journal of Multidisciplinary Healthcare*, Available: <https://pmc.ncbi.nlm.nih.gov/articles/PMC10676677/>
- [46] D. Dasgupta, F. Nino, D. Garrett, K. Chaudhuri, S. Medapati, and A. Kaushal, "A Multiobjective Evolutionary Algorithm For The Task Based Sailor Assignment Problem," presented at the GECCO'09, Montreal, Canadá, 2009.
- [47] C. A. C. Coello, D. A. V. Veldhuizen, and G. B. Lamont, *Evolutionary Algorithms for Solving Multi-Objective Problems*, Second ed. (Genetic and Evolutionary Computation Series). New York: Springer, 2007, p. 800.
- [48] E. R. a. K. Knight, "Inteligencia Artificial," 1994.
- [49] S. Kirkpatrick, C. D. Gelatt, and M. P. Vecchi, "Optimization by Simulated Annealing," *Science*, vol. 220, p. 10, 13 mayo 1983 1983.
- [50] L. S. Pitsoulis and M. G. C. Resende, "Greedy Randomized Adaptive Search Procedures," *AT&T Labs Research Technical Report*, p. 20, 18 enero 2001 2001.

- [51] F. Glover, "Future Paths for Integer Programming and Links To Artificial Intelligence," *Computers & Operations Research*, vol. 13, no. 5, p. 17, 1986.
- [52] S. Ólafsson, *Handbook on Simulation: Metaheuristics* (Handbooks in Operations Research and Management Science VII). Iowa State University, 2006.
- [53] A. Haidine and R. Lehnert, "Multi-Case Multi-Objective Simulated Annealing (MC-MOSA): New Approach to Adapt Simulated Annealing to Multi-objective Optimization," *International Journal of Information Technology*, vol. 4, no. 3, p. 9, 2008.
- [54] J. A. N. González, ""Herramienta para la formación de múltiples equipos de proyecto de desarrollo de software"," Diploma Technical, Centro de Estudios de Ingeniería y Sistemas(CEIS), Universidad Tecnológica de La Habana "José Antonio Echeverría", La Habana, 2018.
- [55] C. Dorn, F. Skopik, D. Schall, and S. Dustdar, "Interaction mining and skill-dependent recommendations for multi-objective team composition," *Data & Knowledge Engineering*, vol. 70, no. 10, pp. 866-891, julio 2011 2011.
- [56] A. Farasat and A. Nikolaev, "Social structure optimization in team formation," *Computers & Operations Research*, vol. 74, no. 2016, pp. 127-142, abril 2016 2016.
- [57] E. L. Ulungu and J. Teghem, "Multi-objective combinatorial optimization problems: A survey," *Journal of Multi-Criteria Decision Analysis*, vol. 3, no. 2, p. 22, agosto, 1994 1994.
- [58] M. Hapke, A. Jaskiewicz, and R. Slowinski, "Pareto simulated annealing for fuzzy multiobjective combinatorial optimization," *Journal of Heuristics*, vol. 6, no. 3, p. 16, 2000.
- [59] Z. G. Estrada, "Incorporación de nuevos operadores y estrategia de reparación en TeamSoft+, herramienta de apoyo en la formación de equipos," 2021.
- [60] A. Hardesty, J. W. Westerman, R. I. Beekun, J. Z. Bergman, and J. H. Westerman, "Images of God and their role in the workplace," *Journal of Management, Spirituality and Religion*, Article vol. 7, no. 4, pp. 315-333, 2010.
- [61] M. Fowler, "Refactoring: improving the design of existing code.," 2018.
- [62] Refactoring.Guru. (2024, 11/julio). *Refactoring.Guru - Design Patterns and Refactoring*. Available: <https://refactoring.guru/>
- [63] G. f. Geeks. (2025, november 6,). *Refactoring - Introduction and Its Techniques*. Available: <https://www.geeksforgeeks.org/software-engineering/refactoring-introduction-and-its-techniques/>
- [64] A. Tyagi. (2025, november 6). *How to Refactor Complex Codebases – A Practical Guide for Devs*. Available: <https://www.freecodecamp.org/news/how-to-refactor-complex-codebases/#heading-techniques-for-refactoring-complex-codebases>
- [65] E. Team. (2025, november 6,). *Software business models explained*. Available: <https://www.embroker.com/blog/software-business-models/>
- [66] I. S. Stephanie Susnjara. (november 6,). Available: <https://www.ibm.com/think/topics/hybrid-cloud-management>
- [67] D. E. Paul Kirvan. (2025, november 6). *What is product as a service?* Available: <https://www.techtarget.com/searcherp/definition/product-as-a-service>
- [68] Rajkumar. (2025, november 6). *Hybrid Model in Software Engineering*. Available: <https://www.softwaretestingmaterial.com/hybrid-model/>
- [69] J. Plataform, "Enterprise Edition (Java EE)," ed.

- [70] J. I. R. Javier García de Jalón, Iñigo Mingo, Aitor Imaz, Alfonso Brazález, Alberto Larzabal, Jesús Calleja, Jon García, *Aprenda Java como si estuviera en primero*. Campus tecnológico de la universidad de navarra, 2000.
- [71] S. S. M. Pratiksha D Dutonde, Monali Sunil Korvate, Sumangla Bafna, Prof. Dhiraj D Shirbhate "Website Developmemt Technologies: A Review," vol. 10, no. I, pp. 359-366 Available: <https://doi.org/10.22214/ijraset.2022.39839>
- [72] E. A. C. C. J. M. C. Reynaldo, "Migración de TestSoft a una plataforma SaaS con enfoque en la usabilidad y experiencia del usuario ", Facutad de Ingeniería Informática, Universidad Tecnológica de La Habana, 2025.
- [73] Q. B. Ulugbek, "WEB FRONT-END AND BACK-END TECHNOLOGIES IN PROGRAMMING," presented at the THEORETICAL ASPECTS IN THE FORMATION OF PEDAGOGICAL SCIENCES, Asia International University, 2024. Available: <https://doi.org/10.5281/zenodo.10518360>
- [74] R. B. Craig Walls, *Spring in Action* (no. In Action Series). Greenwich: Manning, 2005, p. 473.
- [75] A. Zeichik. (2024, november 6). *What Is Spring Boot?* Available: <https://www.oracle.com/latam/database/spring-boot/>
- [76] M. Bauer, "Hello Framework! A heuristic method for choosing front-end JavaScript frameworks," Independent thesis Basic level, Department of Information Technology, Uppsala University, Disciplinary Domain of Science and Technology, 2021.
- [77] R. Vyas, "Comparative Analysis on Front-End Frameworks for Web Applications," *International Journal For Research in Applied Science and Engineering Technology*, vol. 10, no. VII, pp. 298-303. doi: <https://doi.org/10.22214/ijraset.2022.45260> Available: <https://www.ijraset.com/research-paper/comparative-analysis-on-front-end-frameworks-for-web-applications>
- [78] E. G. B. Kennedy Kambona, Wolfgang De Meuter, "An Evaluation of Reactive Programming and Promises for Structuring Collaborative Web Applications," Software Languages Lab, Vrije Universiteit Brussel, 2013.
- [79] H. M. Faruque, "To Build a Front-End Web Application Which JavaScript Framework is Trending Nowadays," Thesis for the Degree of Masters of Engineering, Department of Global Smart IT Convergence Engineering, University of Ulsan, 2022.
- [80] E. Saks, "JavaScript frameworks: Angular vs React vs Vue," Bachelor's Thesis, Business Information Technology, HAAGA-HELIA: University of Applied Sciences, 2019.
- [81] K. Mann, *JavaServer Faces in Action (In Action series)*. Manning Publications, 2005.
- [82] T. P. G. D. Group, *PostgreSQL 9.5.13 Documentation*. 2018.
- [83] IBM. (2023, 16-03). *JPA (Java Persistence API)*. Available: https://www.ibm.com/support/knowledgecenter/es/SSAW57_8.5.5/com.ibm.websphere.wlp.nd.doc/ae/cwlp_jpa.html
- [84] (2024, 16-03). *What is Object-Relational Mapping (ORM) in DBMS?* Available: <https://www.geeksforgeeks.org/what-is-object-relational-mapping-orm-in-dbms/>
- [85] p. JREBEL, "How to Improve JPA Performance," vol. 2025, ed. perforce JREBEL, 2023.
- [86] Hibernate Entity Manager User Guide, 2005. [Online]. Available.
- [87] G. K. Christian Bauer, *Hibernate in action* (In Action Series). Greenwich, CT: Manning, 2005, p. 431.
- [88] E. Foundation. (2025, 16-03). *EclipseLink*. Available: <https://wiki.eclipse.org/EclipseLink/>

- [89] A. Mankale, *Spring Security 3.x Cookbook*. Packt Publishing, 2015, p. 300.
- [90] S. Solutions. (16-03). *CODE QUALITY AND SECURITY, UNDER YOUR CONTROL*. Available: <https://www.sonarsource.com/products/sonarqube/>
- [91] S. Libres. (2025, Mar 16). *¿Qué es y para qué sirve SonarQube?* . Available: <https://somoslibres.org/index.php/17-nieuws/programacion/13669-que-es-y-para-que-sirve-sonarqube>
- [92] SonarSource. (2023, Mar. 20). *SonarQube Documentation*. Available: <https://docs.sonarqube.org/latest/>
- [93] F. X. Cheng Ding, Gopakumar Gopalakrishnan,Weining Qian,Aoying Zhou, "TeamGen:AnInteractiveTeamFormationSystemBased on Professional Social Network*," 2017.
- [94] IBM. (november 15). *¿Qué es la API REST?* Available: <https://www.ibm.com/es-es/think/topics/rest-apis>
- [95] S. Kumar, "A REVIEW ON CLIENT-SERVER BASED APPLICATIONS AND RESEARCH OPPORTUNITY," *International Journal of Recent Scientific Research in Nursing and Health*, vol. 10, no. 07, pp. 33857-33862Available: <https://recentscientific.com/sites/default/files/14205-A-2019.pdf>
- [96] L. B. Jiménez, "Propuesta de arquitectura de software para el producto “Mis Mejores Cuentos”." Trabajo de Diploma para optar por el título de Ingeniero en Ciencias Informáticas, Universidad de las Ciencias Informáticas, 2011.
- [97] G. Cloud. (november 15). *Guía de diseño de APIs*. Available: <https://docs.cloud.google.com/apis/design?hl=es-419>
- [98] M. G. Dan Nosowitz. (november 15). *¿Qué es un endpoint de API?* Available: <https://www.ibm.com/mx-es/think/topics/api-endpoint>

Anexo 5: Datos de estudiantes utilizados para las pruebas.

Nombre completo	Religión	Grupo de Edad	Nombre completo	Religión	Grupo de Edad
Liset Arias Fonseca	Católico	18	Christian Manuel Lebroc Morgado	Agnóstico	19
Saulio Sebastián Beltrandes Pérez	Católico	18	Alain Hernandez Benavides	Agnóstico	19
Alejandro Aguirre García	Católico	18	Andy Carlos Napoles Perez	Agnóstico	20
Amanda Acevedo Madrazo	Católico	18	Dariana Leiro Olivera	Agnóstico	20
Nicole Alavez Matos	Católico	18	Leynier Ramon Hidalgo Yero	Agnóstico	20
Brenda Silvia Gutierrez Herrero	Católico	18	Katherine Ramirez Hidalgo	Agnóstico	20
David Hechavarría Pérez	Católico	18	Antonio Lamoru Varela	Agnóstico	20
Marco Antonio González Ochoa	Católico	18	Dario Gonzalez Morando	Agnóstico	20
Javier de Jesús Castro Heredia	Católico	18	Anyeleni Lara Santana	Agnóstico	20
Ailen Delgado Castro Quiros	Católico	18	Cesar Denis Pastó Escalona	Agnóstico	20
Daniel Alejandro Corria Montanés	Católico	18	Raúl Alejandro De Jesús León	Agnóstico	20
Elisabel Enriquez García	Católico	18	Ismaray Del Sol Cruz	Agnóstico	20
Aldo Ernesto Cabrera Rodríguez	Católico	18	Yankiel Yong Martinez del Sol Cruz	Testigo de Jehová	20
Alessandro Campos Díaz	Católico	18	Felix Manuel Velazquez Sanchez	Testigo de Jehová	20

Daniel Jesús Castro de la Nuez	Católico	18	Carlos Arturo Rojas Aguero	Testigo de Jehová	20
Fabio Ford Campbell	Bautista	18	Diego Rodriguez Arce	Testigo de Jehová	20
Giselle Blanco Aleida	Bautista	18	Andro Rey Garcell	Testigo de Jehová	20
Carlos Alejandro Fonseca Vilar	Bautista	18	Alberto Rodriguez Castro	Testigo de Jehová	21
Naylet Benitez Velázquez	Bautista	18	Diego Basilio Zada Fernandez	Testigo de Jehová	21
Daniel Alejandro Boza Montano	Bautista	18	Dairo Dasiel Vilella Duran	Testigo de Jehová	21
Adrián Baez Dominguez	Bautista	18	Juan Alberto Trimifo Mendez	Testigo de Jehová	21
Rafael Bartelemi Prada	Bautista	18	Mónica Rodríguez Delgado	Testigo de Jehová	21
Fernando Eloy Bermúdez León	Bautista	18	Adolfo Valdés Rizo	Testigo de Jehová	21
Ernesto Alejandro Carralero Conde	Bautista	18	Sandro Torriente Castafio	Testigo de Jehová	21
Kevin Carrera Calzado	Bautista	18	Julio César Polo Mesa	Testigo de Jehová	21
Ever Azcuy Saborit	Bautista	18	Alberto Macias Agüero	Testigo de Jehová	21
Kristian Carreño Contino	Bautista	18	Deynerd Pedraza Chinaea	Testigo de Jehová	21
Denilson Carmona Machado	Bautista	18	Fabian Alejandro Reyes Montero	Judaísmo	18
Dashiel Beltrán Varela	Bautista	18	René Duran Losada	Judaísmo	18

Diego Alfonso Criado	Bautista	18	Sofía Álvarez Cisneros	Judaísmo	18
Juliette García Camps	Pentecostal	18	Dayron Álvarez Tudela	Judaísmo	18
Alberto José Hernández Barreiro	Pentecostal	18	Claudia González Fernández	Judaísmo	18
Anabel Gil Llorente	Pentecostal	18	Daniel José Batiz González	Judaísmo	18
Alejandro García Rodríguez	Pentecostal	18	Hans Concepción Álvarez	Judaísmo	18
Rocío Fernández Carrero	Pentecostal	18	Jesús Manuel Castellanos Reynaldo	Judaísmo	18
Osvaldo David García Reyes	Pentecostal	18	Rafael Castellanos Velázquez	Judaísmo	18
Eduardo Alejandro González Martell	Pentecostal	18	Anyelys Mariam De la Paz Montes	Judaísmo	18
Luis Emilio Llorens Couso	Pentecostal	18	Oscar Dixon Hernández	Judaísmo	18
Waldo Fernández Farinas	Pentecostal	18	Kevin Gallardo González	Judaísmo	18
Gabriel Fernández Lorenzo	Pentecostal	18	Joan Pablo González Miranda	Judaísmo	18
Alejandro Estrada Suárez	Pentecostal	18	Jimmy Peña García	Judaísmo	18
Lino Alejandro Criado González	Pentecostal	18	Silvio Abel Hernández López	Judaísmo	18
Marcos Alejandro Fernandez Pifiero	Pentecostal	18	Andy Daniel González Guevara	Yoruba	19
Camila María Martín Garrido	Pentecostal	18	Alfredo Hernández Rodríguez	Yoruba	19

Lilian Rosa Rojas Rodriguez	Pentecostal	18	Kendry Owen Estévez Rodríguez	Yoruba	19
Daniel Pupo Carmenate	Ateo	18	Laura Estefany Prieto Fernández	Yoruba	19
Miguel Pérez Orue	Ateo	18	Alejandro Montes Hernández	Yoruba	19
Alejandra Sanchez Cabrera	Ateo	18	Ernesto Matamoros Fernández	Yoruba	20
Luis Javier Perez Ruiz	Ateo	18	Frank Mesa Domínguez	Yoruba	20
Melany Rodriguez Arias	Ateo	18	Patricia Barrios Pupo	Yoruba	20
Daniel Zuazo Tejero	Ateo	19	Daniel Castro Hernández	Yoruba	20
Carlos Alberto Puyalena Placeres	Ateo	19	Laura Mary Rosales Domínguez	Yoruba	20
Lorena Robinson Garcia	Ateo	19	Adrián Suárez Padrón	Yoruba	20
Alex Padin Perez	Ateo	19	Ariel Armando Vargas Saucedo	Yoruba	20
Eva Ercilia Vazquez Garcia	Ateo	19	Amaury Daniel Venero Roque	Yoruba	20
Karla Quintela Fernandez	Ateo	19	Cesar Velázquez Martínez	Yoruba	21
Brian Serrano Alfonso	Ateo	19	Daniel Mosquera Molina	Yoruba	21
Cristian Paez Olcha	Ateo	19	Andy Peláez Martínez	Yoruba	21
Jacqueline Menendez de San Pedro Guillen	Ateo	19	Maria Karla Marfalane Delgado	Yoruba	21

Adriana Perez Serafin	Ateo	19	Lissandra María Martínez Llorente	Yoruba	21
Ray Maestre Peña	Agnóstico	19	Elena Rita González Llorente	Yoruba	21
Miguel Angel Lopez Garcia	Agnóstico	19	Julio Sergio López Quirós	Yoruba	21
Jose Janquiel Lopez Rondon	Agnóstico	19	Eduardo Enrique Fonseca Heredia	Yoruba	21
			Daniel Alejandro Daine Rodríguez	Yoruba	21